



Univerzitet u Novom Sadu
**FAKULTET TEHNIČKIH NAUKA U
NOVOM SADU**



Luka Đukić

Razvoj aplikacije za realizaciju prenosa sredstava između blokčejn mreža

Diplomski rad
- Osnovne akademske studije -

Novi Sad, 2025.



UNIVERZITET U NOVOM SADU • FAKULTET TEHNIČKIH NAUKA
21000 NOVI SAD, Trg Dositeja Obradovića 6

KLJUČNA DOKUMENTACIJSKA INFORMACIJA

Redni broj, RBR :		
Identifikacioni broj, IBR :		
Tip dokumentacije, TD :		Monografska dokumentacija
Tip zapisa, TZ :		Tekstualni štampani materijal
Vrsta rada, VR :		Diplomski rad
Autor, AU :		Luka Đukić
Mentor, MN :		dr Dušan Gajić, vanredni profesor
Naslov rada, NR :		Razvoj aplikacije za realizaciju prenosa sredstava između blokčejn mreža
Jezik publikacije, JP :		Srpski / latinica
Jezik izvoda, Jl :		Srpski
Zemlja publikovanja, ZP :		Republika Srbija
Uže geografsko područje, UGP :		Vojvodina
Godina, GO :		2025
Izdavač, IZ :		Autorski reprint
Mesto i adresa, MA :		Novi Sad; trg Dositeja Obradovića 6
Fizički opis rada, FO : (poglavlja/strana/ citata/tabela/slika/grafika/priloga)		7/47/0/1/5/0/23
Naučna oblast, NO :		Elektrotehnika i računarstvo
Naučna disciplina, ND :		Primenjeno softversko inženjerstvo
Predmetna odrednica/Ključne reči, PO :		Distribuirani sistemi, blokčejn, algoritmi konsenzusa
UDK		
Čuva se u, ČU :		U biblioteci Fakulteta tehničkih nauka, Novi Sad
Važna napomena, VN :		
Izvod, IZ :		U ovom radu predstavljene su osnove distribuiranih sistema i njihova kategorizacija. Dat je sažet prikaz načina funkcionisanja blokčejn sistema, njihovih generacija i različitih algoritama konsenzusa. Praktični deo rada obuhvata opis prototipa aplikacije za realizaciju prenosa sredstava između blokčejn mreža prvog i drugog sloja (most). Na kraju rada iznete su kritike i sugestije za budući napredak kroz zaključak.
Datum prihvatanja teme, DP :		
Datum odbrane, DO :		
Članovi komisije, KO :	Predsednik:	dr Dinu Dragan, vanredni profesor
	Član:	dr Veljko Petrović, docent
	Član, mentor:	dr Dušan Gajić, vanredni profesor
		Potpis mentora



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual printed material
Contents code, CC :	Master Thesis
Author, AU :	Luka Đukić
Mentor, MN :	Dušan Gajić, Associate Professor, Ph. D
Title, TI :	Development of an application for transacting funds between blockchain networks
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian
Country of publication, CP :	Republic of Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2025
Publisher, PB :	Author's reprint
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/47/0/1/5/0/23
Scientific field, SF :	Electrical Engineering
Scientific discipline, SD :	Computer Engineering, Engineering of Computer Based Systems
Subject/Key words, S/KW :	Distributed systems, blockchain, consensus algorithms
UC	
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad, Serbia
Note, N :	
Abstract, AB :	This paper presents the fundamentals of distributed systems and their categorization. It provides a concise overview of the functioning of blockchain systems, their generations, and different consensus algorithms. The practical part of the paper describes a prototype application for the realization of asset transfers between first- and second-layer blockchain networks (bridge). Finally, the paper presents critiques and suggestions for future improvements in the conclusion
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	
Defended Board, DB :	President: Dinu Dragan, PhD, Associate Professor
	Member: Veljko Petrović, PhD, Assistant Professor
	Member, Mentor: Dušan Gajić, PhD, Associate Professor
	Mentor's sign

	UNIVERZITET U NOVOM SADU • FAKULTET TEHNIČKIH NAUKA	Datum:
	21000 NOVI SAD, Trg Dositeja Obradovića 6	
	ZADATAK ZA IZRADU DIPLOMSKOG (BACHELOR) RADA	List/Listova:
		/

(Podatke unosi predmetni nastavnik - mentor)

Vrsta studija:	☒ Osnovne akademske studije ☐ Osnovne strukovne studije
Studijski program:	Primenjeno softversko inženjerstvo
Rukovodilac studijskog programa:	dr Aleksandar Selakov, vanredni profesor

Student:	Luka Đukić	Broj indeksa:	PR3/2021
Oblast:	Elektrotehničko i računarsko inženjerstvo		
Mentor:	dr Dušan Gajić, vanredni profesor		

NA OSNOVU PODNETE PRIJAVE, PRILOŽENE DOKUMENTACIJE I ODREDBI STATUTA FAKULTETA IZDAJE SE ZADATAK ZA DIPLOMSKI (Bachelor) RAD, SA SLEDEĆIM ELEMENTIMA:

- problem – tema rada;
- način rešavanja problema i način praktične provere rezultata rada, ako je takva provera neophodna;
- literatura

NASLOV DIPLOMSKOG (BACHELOR) RADA:

**RAZVOJ APLIKACIJE ZA REALIZACIJU PRENOSA SREDSTAVA IZMEĐU
BLOKČEJN MREŽA**

TEKST ZADATKA:

- Proučiti osnovne koncepte i mehanizme rada blokčejn sistema
- Upoznati se sa tehnikom prenosa sredstava između različitih blokčejn mreža putem mostova
- Implementirati prototip aplikacije za prenos sredstava između različitih blokčejn mreža putem mostova
- Analizirati realizovanu implementaciju i izvesti najvažnije zaključke

Rukovodilac studijskog programa:	Mentor rada:

Primerak za: ☐ - Studenta; ☐ - Mentora



UNIVERZITET U NOVOM SADU • **FAKULTET TEHNIČKIH NAUKA**
21000 NOVI SAD, Trg Dositeja Obradovića 6

IZJAVA O NEPOSTOJANJU SUKOBA INTERESA

Izjavljujem da nisam u sukobu interesa u odnosu mentor – kandidat i da nisam član porodice (supružnik ili vanbračni partner, roditelj ili usvojitelj, dete ili usvojenik), povezano lice (krvni srodnik mentora/kandidata u pravoj liniji, odnosno u pobocnoj liniji zaključno sa drugim stepenom srodstva, kao ni fizičko lice koje se prema drugim osnovama i okolnostima može opravdano smatrati interesno povezanim sa mentorom ili kandidatom), odnosno da nisam zavisana od mentora/kandidata, da ne postoje okolnosti koje bi mogle da utiču na moju nepristrasnost, niti da stičem bilo kakve koristi ili pogodnosti za sebe ili drugo lice bilo pozitivnim ili negativnim ishodom, kao i da nemam privatni interes koji utiče, može da utiče ili izgleda kao da utiče na odnos mentor-kandidat.

U Novom Sadu, dana _____

Mentor

Kandidat

Sadržaj

1	Uvod	4
2	Teorijske osnove.....	5
2.1	Distribuirani računarski sistemi	5
2.1.1	Ciljevi distribuiranog računarskog sistema	5
2.1.2	Tipovi distribuiranih računarskih sistema	10
2.1.3	Tipovi softverske arhitekture.....	12
2.1.4	Tipovi systemske arhitekture	13
2.2	Kriptografija.....	16
2.3	Blokčejn	17
2.3.1	Problem	18
2.3.2	DLT	19
2.3.3	Struktura bloka	19
2.3.4	Algoritmi konsenzusa.....	21
2.3.5	Generacije blokčejna	26
3	Opis korišćenih tehnologija	29
3.1	Programski jezik Solidity.....	29
3.2	Ganache	30
3.3	Programske biblioteke web3.py i web3.js	30
3.4	Flask.....	31
3.5	React	31
4	Opis softverskog rešenja.....	32
4.1	Arhitektura sistema	32
4.2	Pametani ugovori	32
4.3	Back-end aplikacija.....	37
4.3.1	Rukovanje pametnim ugovorom	37
4.3.2	Web server.....	42
4.4	Izgled korisničkog interfejsa.....	43
5	Zaključak	45
6	Literatura	47
7	Biografija	49

1 Uvod

Internet je spojio čitav svet. Informacije, roba i usluge danas putuju brže nego ikada, a granice između država i kontinenata gotovo da više ne predstavljaju prepreku. Ipak, uprkos globalnoj povezanosti, nije u ljudskoj prirodi da bezrezervno veruje nepoznatim osobama, organizacijama ili institucijama koje su fizički i kulturološki udaljene. Zbog toga živimo u vremenu u kojem je poverenje postalo ograničen resurs — kako u ljude, a to više u velike korporacije i globalne institucije.

Upravo iz tog razloga je proistekla potreba za tehnologijom koja omogućava saradnju bez posrednika, na temelju matematičke dokazivosti umesto institucionalnog autoriteta. Tu se pojavljuje blokčejn tehnologija — inovacija koja je ljudima ponudila mogućnost decentralizacije, nezavisnosti od banaka, velikih korporacija i međunarodnih finansijskih institucija. Svaki pojedinac može učestvovati u sistemu u meri u kojoj to želi, a poverenje u mrežu zasniva se na kriptografskoj zaštiti, otvorenom kodu i transparentnom konsenzusu koji matematički dokazuju ispravnost i neporecivost svih zapisa.

Ipak, kao i svaki drugi kompleksni sistem, blokčejn ima svoja ograničenja. Jedno od ključnih je skalabilnost, odnosno ograničena mogućnost da se jedna mreža efikasno nosi sa rastućim brojem korisnika i transakcija. Kao odgovor na taj izazov razvijen je čitav ekosistem nezavisnih mreža — *Bitcoin, Ethereum, Solana, Polygon, Avalanche, Base* itd... Svaka pokušava da pronađe ravnotežu između brzine, sigurnosti i decentralizacije.

Međutim, pojava više mreža donela je novi problem — fragmentaciju sistema. Svaka mreža funkcioniše kao zaseban sistem sa sopstvenim pravilima i tokenima, a međusobna komunikacija između njih gotovo da ne postoji. Ova nepovezanost blokčejn mreža ograničava razvoj složenijih decentralizovanih aplikacija (dApps).

Kako bi se taj problem prevazišao, počeli su da se razvijaju mostovi između blokčejnova (blockchain bridges) — aplikativni protokoli koji omogućavaju prenos podataka i sredstava sa jedne mreže na drugu. Ovi mostovi predstavljaju ključni korak ka stvaranju povezanog blokčejn ekosistema.

Upravo iz tog razloga, tema ovog rada jeste razvoj prototipa aplikativnog mosta između blokčejn mreža, sa ciljem da se prikaže kako različiti sistemi mogu međusobno komunicirati i razmenjivati vrednost na siguran, transparentan i efikasan način.

2 Teorijske osnove

Od početka komercijalnog računarstva do danas, udaljenost računara od krajnjeg korisnika predstavlja ustaljen problem. U početku su računari (*Main frame*) bili isuviše skupi da bi ih obični korisnici nabavljali. Taj problem se smanjio napretkom tehnologije, izumom PN spoja i poluprovodničke tehnologije, koji je ubrzao razvoj računara, njihovih performansi, a smanjio njihovu veličinu i cenu. Vremenom su korisnici došli u situaciju da mogu sebi da priušte vlastini (personalni) računar (IBM PC 1981. godina [12]). Međutim, vreme korišćenja vlastitih resursa nije dugo potrajalo. Razvojem računarskih mreža, email-a, interneta, komunikacija računara dolazi u prvi plan. Nedugo zatim, opet je počela era dominacije super računara, ovog puta u formi servera u oblaku (*Cloud*). Većina obrade podataka opet se preselila na udaljeni računar, a lični računari služe većinski za prikaz već obrađenih podataka, dobijenih u komunikaciji preko računarske mreže.

2.1 Distribuirani računarski sistemi

Informacioni sistem predstavlja hardver, komunikacionu opremu i softver za prikupljanje, kreiranje, obradu i skladištenje podataka.

Distribuirani računarski sistem je skup nezavisnih računara koji svojim korisnicima deluje kao jedinstven i koherentan sistem.

Ova definicija naglašava dva ključna aspekta distribuiranog sistema: Prvo, čine ga više računara koji su nezavisni jedni od drugih, što znači da bi svaki *per se* mogao da funkcioniše bez ostalih (ima svoju memoriju i svoje napajanje). Drugo, korisnik vidi sistem kao jednu celinu, što znači da moraju da komuniciraju i budu, bar delom, sinhronizovani. Definicija ne određuje vrstu računara ili način njihovog umrežavanja, tako da dozvoljava izuzetno heterogenu opremu i komunikaciju.

2.1.1 Ciljevi distribuiranog računarskog sistema

Da bi se isplatilo pravljenje distribuiranog računarskog sistema, koji je u opštem slučaju znatno složeniji od centralizovanog sistema, on mora da, u najvećem mogućem stepenu zadovolji četiri osnovna cilja svih distribuiranih sistema, to su: povezanost, transparentnost (distribuiranost), otvorenost i skalabilnost.

2.1.1.1 Povezanost

Primarni cilj distribuiranog sistema je da omogući povezivanje udaljenih resursa, korisnika i aplikacija. Razloga može biti više: Ekonomičnije je deliti jedan uređaj, pogotovo ako je skup,

nego nabavljati više sličnih primeraka. Takvi uređaji mogu biti prosti kancelarijski: štampači, skeneri, ali i skupi računari: super-računari, visoko performantna skladišta podataka i sl.

Drugi razlog jeste olakšavanje saradnje različitih aplikacija i ljudi. Primeri toga su onlajn konferencije i pozivi poput Google Meet-a, Discord-a i Skype-a, deljeni sistemi datoteka, koji omogućuju deljene audio, video, tekstualnih i drugih datoteka poput Google Disk-a i sl, elektronska pošta koja omogućuje brzu razmenu poruka kao što su Google Mail, Yahoo, Hotmail i dr.

Treći razlog je povezivanje računara zarad poboljšanja performansi pri obradi podataka, poput klastera. Većina današnjih servera su klasteri, formirani umrežavanjem velikog broja računara relativno skromnih mogućnosti. Ovaj pristup omogućava izuzetno veliki nivo paralelizacije obrade, što znatno poboljšava performanse sistema.

U određenim infrastrukturnim sistemima, povezanost omogućava nadzor i upravljanje na daljinu. Primeri toga u elektorenergetskom sistemu su: praćenje napona i pada napona na sabirnici, pa kao reakcija na to upravljačka naredba za promenu pozicije teretnog menjača.

2.1.1.2 Transparentnost

Distribuirani sistem je transparentan kada ga njegovi korisnici doživljavaju kao jedan sistem, tj. on od svojih korisnika skriva (apstrahuje) svoju složenost.

Postoji više različitih vrsta transparentnosti datih u tabeli ispod.

Vrsta transparentnosti	Opis
Transparentnost pristupa	Skriva razliku u reprezentaciji podataka i pristupa resursima
Lokacijska transparentnost	Skriva razliku u lokaciji resursa
Migracijska transparentnost	Skriva mogućnost stalne promene lokacije resursa
Relokacijska transparentnost	Skriva mogućnost promene lokacije resursa u upotrebi
Replikacijska transparentnost	Skriva mogućnost pravljenja replika resursa
Transparentnost konkurentnosti	Skriva konkurentnost pristupa resursima
Transparentnost otkaza	Skriva otkaze sistema

Tabela 1. Vrste transparentnosti

Transparentnost pristupa skriva od korisnika interni način reprezentacije podataka. Na najnižem nivou primer toga jeste razlika između Little Endian i Big Endian sistema, koji imaju drugačiji raspored bajtova višebajtnih podataka (U Little Endian sistemima se na najnižoj adresi

u memoriji čuva bajt sa najmanje značajnim bitima, a kod Big Endian sistema obrnuto), ali se pri slanju podataka na mrežu svi poravnavaju da budu Big Endian. Drugi primer bi bila različita konvencija za nazivanje datoteka različitih operativnih sistema.

Sledeće četiri vrste transparentnosti odnose se na lokaciju resursa, ali pod različitim uslovima. Lokacijska transparentnost odnosi se na skrivanje lokacije resursa. Što znači da korisnik ne treba da zna gde se konkretno nalazi resurs koji koristi, na primer na kom serveru se nalazi datoteka sa muzikom korisnika na Google Disk-u. Migracijska transparentnost se odnosi na skrivanje trajnog premeštanja podataka. Relokacijska transparentnost predstavlja skrivanje promene lokacije podataka prilikom upotrebe. Relokacija se vrši radi poboljšanja performansi, jer što je resurs bliže, brže mu se pristupa. Tehnika približavanja resursa korisniku u računarstvu se naziva keširanje (*caching*). Primer ovoga je način na koji CDN-ovi (*Content Delivery Network*) funkcionišu. Naime resurs, najčešće video dataoteka, se u početku nalazi daleko na korenskom čvoru. Kada korisnik zatraži pomenuti resurs, on biva pomeren na server bliže korisnika. Na taj način se vreme pristupa korisnika smanjuje, ali je i vreme pristupa sledećeg korisnika, koji je blizu prvog, smanjuje. Replikacijska transparentnost skriva od korisnika mogućnost postojanja replike, postupka njenog kreiranja, i redovnog ažuriranja, kao i, u slučaju ispada glavnog resursa (*master*), prelazak na rad sa replikom. Pogotovo složen problem predstavlja poravnavanje izuzetno udaljenih replika (pr. replike na različitim kontinentima). Za ovaj problem se uvode novi modeli konzistencije poput konačne konzistencije (*Eventual Consistency*), koji prevazilaze obim ovog rada. Sve prethodne četiri vrste transparentnosti se mogu lakše zadovoljiti uvođenjem logičkih imena. Na taj način, logičko ime skriva lokaciju, eventualnu migraciju, relokaciju ili replikaciju resursa. Najpoznatiji primer logičkih imena je URL sistem (*Uniform Resource Locator*). On odvaja ime resursa od putanje do njega. Ta putanja ipak mora biti izračunata, te se razvija poseban distribuiran sistem za njeno razrešavanje – DNS (*Domain Name System*).

Transparentnost konkurentnosti se bavi skrivanjem konkurentnosti od korisnika, tj. jednovremenog prisutpa istom resursu od strane raznih korisnika sistema. Postoje dva osnovna mehanizma konkurentnosti u distribuiranim sistemima: zaključavanje resursa – resurs se zaključa dok ga koristi određeni korisnik, pa na taj način on ima ekskluzivna prava nad resursom. Problem nastaje ako se desi otkaz bilo kog dela sistema, pa resurs ostane neoslobođen, tj. zaključan. Drugi mehanizam konkurentnosti jesu transakcije – atomične jedinice komunikacije, zahtevi, koji su ili celi ispunjeni ili celi odbačeni. Problem transakcija u distribuiranom sistemu je složenost njihove implementacije (algoritmi poput 2PC i 3PC koji prevazilaze obim ovog rada).

Transparentnost otkaza je skrivanje otkaza delova sistema od korisnika i mogućnost oporavka od istog. Naime, u distribuiranom sistemu sa mnogo heterogenih resursa, čak i uz veliku pouzdanost, velika je verovatnoća da će u svakom trenutku bar deo sistema biti u otkazu. Osobina

dobrog distribuiranog sistema je da nastavi sa obradom uprkos otkazima pojedinačnih delova sistema. Tako distribuiran sistem treba da izbegava da ima kritičnu tačku otkaza (*Single Point of Failure*), kao i da ima replike koje se redovno, po mogućnosti što češće, ažuriraju. Problem otkaza je što sistem ne može da razlikuje otkaz određene komponente od toga da je komponenta iznimno spora (pr. zagušenje).

2.1.1.3 Otvorenost

Otvoren distribuirani sistem pruža svojim korisnicima usluge po standardnim sintaksnim i semantičkim pravilima.

Servisi se obično definišu putem interfejsa. Sintaksu interfejsa predstavlja naziv pruženih usluga interfejsa (funkcija), kao i broj, tip i naziv argumenata i broj, tip i naziv povratnih vrednosti. Ona se obično definiše pomoću namenskog jezika za opis interfejsa IDL (*Interface Definition Language*).

Za razliku od sintakse interfejsa, semantika se ne može opisati formalnim jezicima, već se pruža u neformalnom obliku putem dokumentacije.

Dobro definisan interfejs je kompletan i neutralan. Kompletnost je karakteristika interfejsa kojom on specificira sve podatke neophodne za implementaciju datog interfejsa. Neutralnost interfejsa predstavlja karakteristiku interfejsa da on ne upućuje ni na jednu implementaciju.

Ovako definisan interfejs može biti implementiran od strane raznih komponenti na razne načine i pruža korisniku interfejsa apstrakciju njegove unutrašnje složenosti. Interfejs koji je kompletan i neutralan je pogodan što se tiče portabilnosti i interoperabilnosti.

Interoperabilnost je pojava saradnje komponentni raznih proizvođača oslanjajući se isključivo na međusobne usluge koje su definisane standardom (interfejsom). Portabilnost je stepen prenosivosti aplikacije sa jednog distribuiranog sistema na drugi, koji implementiraju iste interfejse.

Otvorenost se takođe ogleda u mogućnosti zamene komponenti sistema bilo u slučaju ispada, ili u slučaju poboljšanja karakteristika sistema.

2.1.1.4 Skalabilnost

Skalabilnost je mogućnost proširenja distribuiranog računarskog sistema na održiv način.

Proširivanje sistema može biti u tri dimenzije. To su: skalabilnost po veličini, geografska skalabilnost i administrativna skalabilnost. Često sistem koji je skalabilan po bilo kojoj od ovih osi gubi na performansama, to je pojava koju treba nastojati minimizirati.

Skalabilnost po veličini odnosi se na mogućnost dodavanja novih resursa i novih korisnika sistema.

Geografska skalabilnost je ona u kojoj resursi i korisnici mogu biti geografski prilično udaljeni.

Administrativna skalabilnost je mogućnost upravljanja sistemom koji se proteže u raznim administrativnim zonama (raznim regionima, državama).

Na svakoj od ovih pravaca skalabilnosti, nalaze se razni izazovi. Skalabilnost po veličini boluje od problema centralizacije. Bilo da ta centralizacija nastaje od centralizovanog servisa, centralizovanih podataka ili centralizovanog algoritma. Problem centralizovanog servisa sastoji se od toga da određene usluge u distribuiranom sistemu pruža jedan fizički server. On u tom slučaju lako može postati usko grlo za performanse. Povećavanjem broja korisnika tog servera, on se usporava. Čak iako server sam ne predstavlja problem, do zagušenja može doći u komunikacionoj infrastrukturi oko njega. Problem centralizovanih podataka se lako ilustruje na primeru centralizovanog sistema koji koristi jednu bazu podataka, skladištenu na jednom serveru. Tada sve komponente moraju da komuniciraju sa datom bazom podataka, i time moguće preopterećuju bilo bazu podataka direktno, bilo komunikacionu infrastrukturu oko nje, te dolazi do pada performansi. Krajnji problem je centralizovani algoritam. Centralizovani algoritam je onaj koji se bazira na poznavanju topologije celog grafa (Ako distribuirani sistem predstavimo kao graf u kome čvorovi predstavljaju resurse, a grane komunikacionu infrastrukturu koja ih povezuje). Ovi algoritmi nisu efektivni za velike distribuirane sisteme, jer zahtevaju previše komunikacije između komponenti sistema. Algoritmi u distribuiranim računarskim sistemima treba da nastoje da budu decentralizovani. Glavne karakteristike decentralizovanih algoritama su:

1. Nepoznavanje kompletne topologije grafa
2. Čvorovi donose odluke na osnovu lokalnih podataka
3. Otkaz pojedinačnog čvora ne narušava algoritam
4. Nepostojanje centralizovanog sata

Problemi geografske skalabilnosti su sinhrona komunikacija i komunikacija „tačka-tačka“. Prilikom izrade sistema korišćenjem lokalne mreže, vreme utrošeno na komunikaciju je malo, zbog blizine resursa koji komuniciraju i zbog velike brzine LAN-ova (*Local Area Network*). Takve pretpostavke nisu, u opštem slučaju, istinite prilikom geografskog skaliranja distribuiranog sistema. Takođe, određeni mehanizmi funkcionišu na lokalnoj mreži zbog mogućnosti emitovanja poruka (zahtev se emituje svim ostalim resursima u lokalnoj mreži, a resurs koji je zadužen za obradu će poslati odgovor). Ovakav mehanizam je nezamisliv na GAN-u (*Global Area Network* pr. internet).

Problemi administrativnog skaliranja odnose se na poravnavanje razlika između raznih domena po pitanju upotrebe i naplate resursa, njihov upravljanja i nadzora.

Postoje tri osnovna načina prevazilaženja problema pri skaliranju: distribuiranje, repliciranje i skrivanje kašnjenja u komunikaciji. Distribuiranje je tehnika kojom se centralizovanost sistema „razbija“, primer je zamena centralizovanog za decentralizovani algoritam, decentralizovanje centralizovanih servisa na više fizičkih servera i sl. Replikacija pomaže pri problemima centralizovanih servisa i podataka i problemu udaljenosti korisnika od traženog resursa. Još jedno rešenje za problem usporavanja komunikacije je zamena sinhronog za asinhron režim komunikacije, u kojem klijent (strana koja zahteva obradu) ne blokira čekajući na odgovor servisa (strane koja opslužuje), već nastavlja sa korisnim radom, a naknadno, po dobijanju odgovora, ga obradi i prikaže. Ovaj režim komunikacije, doduše, nije odgovarajuć za sve aplikacije (pr. interaktivne aplikacije).

2.1.2 Tipovi distribuiranih računarskih sistema

Svi distribuirani računarski sistemi mogu se podeliti po svojoj nameni u neku od tri sledeće kategorije: distribuirani sistemi za intenzivno računanje, distribuirani informacioni sistemi i distribuirani rasplinuti sistemi [15].

2.1.2.1 Distribuirani sistemi za intenzivno računanje

Ideja distribuiranih sistema za intenzivno računanje je jednostavna. Povezivanjem više računara, kroz mehanizme paralelizacije obrade, pojačava se računarska moć sistema. Unutar ovog tipa postoji dodatna podela na dva podtipa: klastere i gridove.

2.1.2.1.1 Klasteri

Klaster je multiračunar sastavljen od većeg broja radnih stanica, koji se povezuju komercijalnim spoljnim mrežama kao što su ethernet, prekidači (*switch*) i systemske mreže poput SAN-a (*Storage/System Area Network*) [12]. Ideja je jednostavna – kupiti više komercijalnih računara i spojiti ih brzom lokalnom mrežom. Računari unutar ovakvog sistema su homogeni, u značenju da imaju sličan (poželjno isti) hardver i isti operativni sistem. Jedan od računara unutar klastera se proglašava za glavnog, dok su mu ostali podređeni (*slave*). Glavni računar je zadužen za raspodelu posla, nadzor i upravljanje nad njemu podređenim računarima. Takođe, on je jedini računar koji komunicira sa spoljašnosti klastera i pruža interfejs korisnicima. Glavni računar je opremljen bibliotekama za paralelnu obradu, što je suštinski jedini središnji sloj (*middleware*) koji ovakva konfiguracija zahteva. Podređeni računari obično nisu opremljeni nikakvom dodatnom programskom podrškom. Ovakva konfiguracija je karakteristična za *Beowulf*¹ klastere bazirane

¹ *Beowulf* je prvobitno bio specifičan klaster koji su Tomas Sterling i Donald Beker napravili u NASA-i 1994. godine i nazvali po starom engleskom epu *Beowulfu*. Ipak, kasnije je postao sinonim za konfiguraciju klastera sličnu originalnoj realizaciji.

na *Linux* operativnim sistemima. Ovde treba napomenuti da *Beowulf*, iako standardna, nikako nije jedina konfiguracija klastera. Ostale konfiguracije poput MOSIX-a, prevalizaze okvire razmatranja ovog rada.

2.1.2.1.2 Gridovi

Suštinska razlika između gridova i klastera jeste u heterogenosti računara unutar grida. Grid je sastavljen od računara raznih proizvođača, raznih mogućnosti, sa raznim operativnim sistemima. Ovi računari su nekada veoma udaljeni jedni od drugih, te i njihovo povezivanje nije uniformno. Kako bi podržao sve moguće razlike između računara unutar grida, središnji sloj mora biti mnogo više razvijen u gridu nego kod klastera. [1] predlažu programsku arhitekturu za grid, koja pored aplikativnog sloja na vrhu i fizičkog sloja na dnu, sadrži središnji sloj koji je podelj u tri pod sloja: sloj povezivanja, sloj resursa i kolektivni sloj.

Eklatantan primer grida jeste *SETI at home* projekat istraživačkog centra u Berkliju, koji je aktivan od 1999. godine.

2.1.2.2 Distribuirani informacioni sistemi

Za razliku od prethodnog tipa, koji je kao cilj imao povećanje procesne moći, distribuirani informacioni sistemi za cilj imaju integrisanje različitih aplikacija u jedinstven informacioni sistem. Možemo razlikovati nekoliko nivoa integracije.

U početku se integracija sastojala od aplikacija koje su se, zajedno sa bazom podataka, izvršavale na nekom serveru, i čije su usluge bile dostupne kroz RPC-ove (*Remote Procedure Call*) aplikacijama unutar istog sistema (klijentima). Na ovom, osnovnom, nivou integracija je podrazumevala i mogućnost obrade više zahteva u okviru jedne transakcije. To znači da klijent može poslati više zahteva koji su upućeni raznorodnim delovima istog sistema, a oni treba da se izvrše transakcijski (ili da se sve izvrše ili da se ne izvrši nijedna). Ovo je bio začetak distribuirane transakcije. Arhitektura distribuirane transakcije i algoritmi njenog funkcionisanja (ranije pomenuti 2PC i 3PC) nisu obuhvaćeni opusom ovog rada.

Kako su se aplikacije usložnjavale, postalo je jasno da ovakva arhitektura nije dovoljno fleksibilna. Tako su se aplikacije sa svojim bazama podataka odvajale jedna od druge i formirale nezavisne komponente. Zbog integracije ovakvih komponenti softverska arhitektura sistema će razviti više različitih modela koji omogućuju komponentama da direktno komuniciraju jedna sa drugom. Takve programske arhitekture šire su obrazložene u poglavlju 2.1.3

2.1.2.3 Distribuirani rasplinuti sistemi

Za razliku od prethodne dve vrste distribuiranih sistema, koji su imali manje-više stalne komponente, rasplinite distribuirane sisteme karakterišu mali, mobilni uređaji, najčešće sa

baterijskim napajanjem. Tehnologija integrisanih kola je vremenom toliko napredovala da je izrada računara bila toliko jeftina, a njihova veličina se toliko smanjila, da su proizvođači počeli da im nalaze primenu i u ostalim kućnim uređajima: beloj tehnici poput frižidera, zamrzivača, veš mašina, šporeta, klima uređaja, usisivača, u pametnim satovima i ostalim svakodnevnim kućnim uređajima. Ovi računari čine čovekovo svakodnevno okruženje i uglavnom imaju samo bežičnu vezu. Mogu biti poprilično različitih mogućnosti i pružati različite usluge. Kod ovakvih uređaja cilj transparentnosti prestaje da bude razmatran, pošto zbog velike mobilnosti ne može biti ni ostvaren. Oni bi trebalo da efektivno otkrivaju uređaje i servise oko sebe (pr. mobilnog telefona koji se automatski preključuje sa jedne na drugu mrežu bez korisnikovih direktnih naredbi). Ovakvi sistemi mogu biti: kućna automatika, elektronski zdravstveni sistemi i senzorske mreže.

2.1.3 Tipovi softverske arhitekture

Softverska komponenta distribuiranog računarskog sistema je modularna jedinica sa jasno definisanim interfejsima koja je zamenjiva unutar svog okruženja [1].

Logička organizacija komponenti, njihovog međusobnog povezivanja, način njihove interakcije i prenosa podataka između njih čini softversku arhitekturu sistema. Mehanizam komunikacije i koordinacije naziva se konektor, a ostvaruje se preko određenog srednjeg programskog sloja. Postoje četiri osnovna stila softverske arhitekture: slojevita arhitektura, objektno orijentisana arhitektura, arhitektura bazirana na podacima i arhitektura zasnovana na događajima.

Slojevita arhitektura organizuje komponente u strogu hijerarhiju slojeva, u kojoj sloj S_n može da komunicira samo sa slojevima S_{n-1} i S_{n+1} s tim što se viši sloj oslanja na usluge nižeg sloja, tj. sloj S_n se oslanja na sloj S_{n-1} i poziva njegove funkcije, dok se na sam sloj S_n oslanja sloj S_{n+1} koji poziva njegove funkcije. Ovakav način komunikacije upućuje na to da zahtevi za obradom mogu da se propagiraju isključivo niz hijerarhiju, dok se rezultati obrade propagiraju uz datu hijerarhiju. Ovakva arhitektura je dominantna u računarskim mrežama (pr. TCP/IP komunikacioni stek)

U objektno orijentisanoj arhitekturi komponente predstavljaju objekte i povezani su slobodno u graf. Mehanizam komunikacije između njih predstavljaju (nekad udaljeni) pozivi procedura. Ovaj stil softverske arhitekture je zanimljiv zbog izuzetne fleksibilnosti, tj. slobode u komunikaciji.

Arhitektura bazirana na podacima ima jednostavnu suštinu. Cilj komunikacije između komponenti je uvek dobavljanje podataka. Tako jedini način komunikacije unutar ovakvog sistema jeste posredan, preko pristupa istim, deljenim, podacima. Primeri ove arhitekture jesu aplikacije zasnovane da rade sa istom bazom podataka ili deljenim sistemom datoteka.

Arhitektura zasnovana na događajima, pak, se razvila iz drugog zapažanja. Naime, jedini povod komunikacije između komponenti jeste promena stanja sistema. Tako, usled promene stanja sistema, komponenta koja je izvršila promenu emituje događaj kojim javlja datu promenu, dok je srednji programski sloj zadužen da propagira promenu do ostalih zainteresovanih komponenti. Kako bi srednji sloj mogao da izvrši svoj zadatak, on treba da održava evidenciju zainteresovanih komponenti za svaki od mogućih tipova događaja. Kako bi se vodila evidencija zainteresovanih komponenti uveden je mehanizam pretplate, kojom se komponente pretplaćuju na događaje koji su im od interesa. Ovakav model komunikacije naziva se i izdavač-pretplatnik model (*publisher-subscriber*). Prednost ovog modela komunikacije jeste što nijedna komponenta ne upućuje na drugu, tj. komponente su referencijalno razdvojene.

Arhitektura deljenog adresnog prostora čini hibridnu arhitekturu između arhitekture zasnovane na događajima i arhitekture bazirane na podacima. Razlikuje se od arhitekture zasnovane na događajima po tome što perzistira poruke. Pošto se poruke u ovom modelu trajno skladište, njegova prednost je što su komponente i vremenski razdvojene, jer ne moraju biti aktivne u isto vreme.

2.1.4 Tipovi sistemske arhitekture

Sistemska arhitektura je realizacija softverske arhitekture, sa određenim komponentama, njihovom interakcijom i rasporedom [1].

Osnovne vrste sistemske arhitekture su: centralizovana, decentralizovana i hibridna.

2.1.4.1 Centralizovane sistemske arhitekture

Centralizovana sistemska arhitektura je ona u kojoj se obrada podataka dešava unutar jednog fizičkog ili virtuelnog računara, koji predstavlja koordinatora u sistemu. Sva interakcija sa sistemom se uvek vrši njegovim posredstvom i svi podaci unutar sistema moraju proći kroz njega.

Server (servis) je proces koji pruža određenu uslugu, tj. određenu obradu podataka i opslužuje korisnike. Klijent je korisnik servera, predstavlja proces koji zahteva obradu. Komunikacija između klijenta i servera odvija se tako što klijent zatraži obradu slanjem zahteva i očekuje odgovor koji server šalje po završetku obrade. Ovakav model komunikacije naziva se klijent-server model. Treba napomenuti da je razlikovanje klijentskog i serverskog procesa moguće samo u specifičnom kontekstu, jer određeni procesi mogu biti klijenti u jednoj, a serveri u drugoj interakciji (pr. *back-end* veb aplikacija je server *front-end* aplikaciji, a klijent bazi podataka).

U centralizovanom sistemu, koordinator sistema je server. Pošto se korisničke aplikacije mogu razdvojiti na tri softverska sloja: perzistenciju, obradu i prezentaciju podataka, nameće se

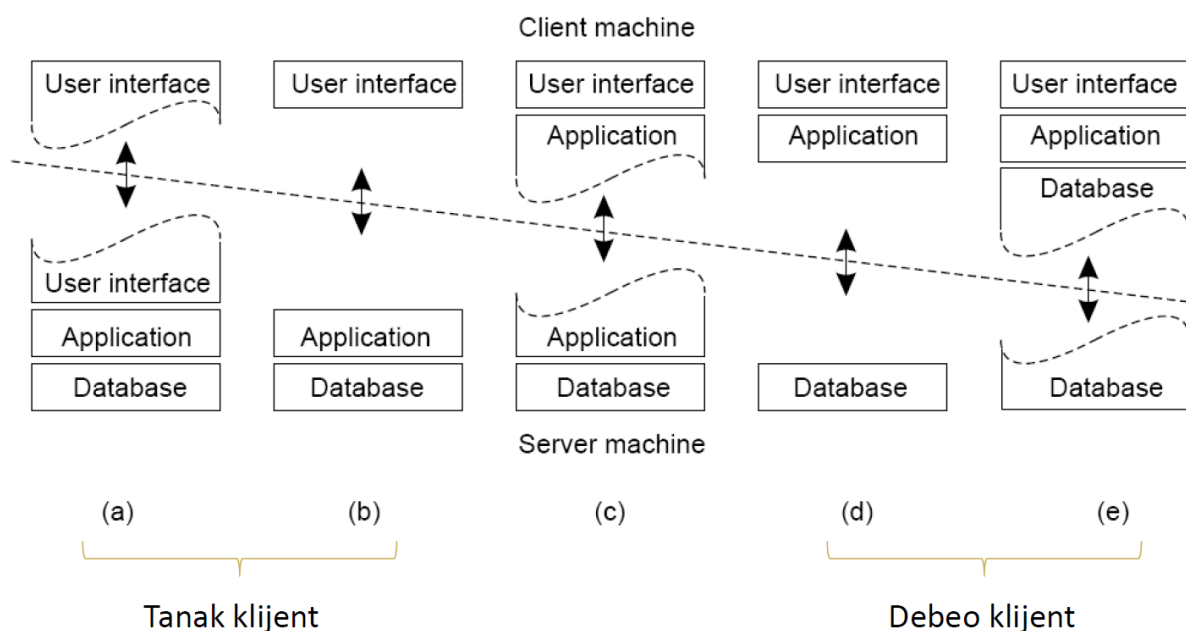
pitanje fizičke, systemske organizacije datih slojeva. Postoje tri pristupa: jednoslojna, dvoslojna i troslojna organizacija.

Jednoslojna organizacija se koristila u interakciju između mejnfrejma i terminala, i danas je već mahom zastarela. U njoj je server (mejnfrejm) bio zadužen za perzistenciju, obradu i prezentaciju podataka, dok je klijent (terminal) samo ispisivao podatke koje mu je server slao.

Dvoslojna organizacija se i danas sreće. Glavno pitanje dvoslojne organizacije je kako podeliti tri softverska na dva fizička sloja. Moguće realizacije date su na slici ispod.

Ako server preuzima veći deo aplikacije (slika a), dok je klijent zadužen samo za prikaz podataka, tu konfiguraciju nazivamo *debeli server*. Ako, pak, preselimo većinu slojeva na klijentsku stranu (slika e), a server postaje zadužen za perzistenciju podataka govorimo o *debelom klijentu*. Obe organizacije imaju svoje prednosti ali i mane, zato je važno prepoznati potrebe pojedinačne aplikacije.

Troslojna organizacija je danas standard i, kao što i naziv govori, čine je tri računara na svaki od kojih je pohranjen po jedan softverski sloj. Ova arhitektura je dominantna u modernim veb aplikacijama. U takvoj arhitekturi se sloj aplikacije (ili moderno posebna aplikacija) koji služi za prezentaciju podataka naziva prednjim delom aplikacije, dok sloj (aplikaciju) zaduženu za obradu podataka nazivamo zadnjim delom aplikacije. Za perzistenciju je zadužena baza podataka.



Slika 2.1 Dvoslojna organizacija
(preuzeto iz [15])

2.1.4.2 Decentralizovane systemske arhitekture

Uzmimo za primer prethodno objašnjenu troslojnu veb aplikaciju. Ona, iako centralizovana, u smislu postojanja jedinstvene tačke otkaza, ipak sadži određen nivo decentralizacije. Ta decentralizacija se sastoji u tome što različiti slojevi date aplikacije mogu biti smešteni na različite

računare. Kada iz tog ugla pogledamo naš sistem, možemo videti da su logički različite komponente smeštene na fizički različite računare – tu pojavu nazivamo vertikalnom distribucijom.

Pored vertikalno distribuiranih sistema, postoje i horizontalno distribuirani sistemi, koji su decentralizovani u užem smislu.

U horizontalno distribuiranim sistemima se softverske komponente, predstavljale one klijent ili server, mogu fizički podeliti na logički ekvivalentne delove, od kojih svaki radi nad delom celokupnih podataka. U ovakvim sistemima sve komponente su ravnopravne, a komunikacija je simetrična, svaka komponenta jednovremeno može vršiti i ulogu servera i ulogu klijenta. Zbog navedene ravnopravnosti, ovakvi sistemi se još i nazivaju P2P sistemi (*Peer to Peer*, *Peer-vršnjak*). Komponente u P2P sistemima se mogu predstaviti čvorovima, dok se kanali njihove međusobne komunikacije mogu predstaviti granama grafa. Ovakav graf u literaturi se naziva *prekrivajuća mreža*, i ključno pitanje projektovanja P2P sistema je upravo način organizovanja prekrivajuće mreže. U prekrivajućoj mreži svaki čvor može komunicirati direktno samo sa svojim susedima (čvor sa kojim je spojen granom), dok komunikacija dva nesusedna čvora mora biti posredna.

Postoje dva pristupa projektovanju prekrivajuće mreže: strukturirani i nestrukturirani. Strukturirana pokrivajuća mreža podrazumeva konvenciju podele podataka između čvorova. Ova konvencija otklanja potrebu za preplavlivanjem, tj. traženjem čvora sa resursom unutar grafa. Najčešća realizacija strukturirane prekrivajuće mreže je neka vrsta distribuirane heš tabele. U ovoj realizaciji, čvorovi i resursi dobijaju identifikator. Konvencija dalje određuje na koji se način od identifikatora resursa može saznati identifikator čvora u kojem se traženi resurs nalazi. Strukturirani pristup omogućuje mnogo brže traženje i pretragu resursa unutar sistema. Primer ovakvog sistema je DNS, koji služi za mapiranje mnemoničnih u IP (*Internet Protocol*) adrese.

Drugi pristup projektovanju prekrivajuće mreže je nestrukturirani. Njegova glavna osobina je što se traženi resurs može nalaziti u bilo kom čvoru unutar sistema. Pretraga ovakvog grafa efektivno se svodi na preplavlivanje (tehnika kojom se upit šalje svim susednim čvorovima, koji šalju upit svim svojim susednim čvorovima itd. kad upit stigne do čvora sa traženim resursom on će istom tehnikom poslati odgovor). Čvorovi u nestrukturiranim sistemima najčešće nasumično određuju skup svojih suseda, sa ciljem da topologija sistema liči na nasumičan graf. U teoriji, nastoji se da svaka dva čvora imaju istu verovatnoću da budu povezani, dok se u praksi to često ne dešava.

Ponekad su P2P sistemi organizovani hijerarhijski, tako čvorovi mogu biti u skupu slabih vršnjaka (engl. *weak peer*) i super vršnjaka (engl. *superpeer*). Super vršnjaci se prema slabim vršnjacima odnose kao serveri prema klijentima, dok se međusobno jedan prema drugom odnose

kao vršnjaci (čine P2P sistem). U ovakvim sistemima super vršnjaci uglavnom imaju ulogu mešetara (*broker*) i posreduju razmeni resursa između slabih vršnjaka i ostataka sistema.

Blokčejn sistemi su najčešće realizovani kao hijerarhijski P2P sistemi. *Puni* čvorovi su oni koji sadrže podatke, dok su *klijenti* čvorovi koji zahtevaju obradu transakcija. Pored ova dva osnovna tipa čvora mogu postojati *lani* čvorovi, koji sadrže deo lanca blokova (zaglavlja blokova ili blokove koji su im od interesa). Takođe, *puni* čvorovi mogu biti validatori (*rudari*) ako učestvuju u validaciji novih blokova.

2.2 Kriptografija

Kriptografija je nauka koja izučava algoritme za transformaciju čitljivih podataka u format koji nije čitljiv i njihovu inverznu transformaciju u osnovni oblik. Kako je kriptografija nauka za sebe, u ovom radu osvrnućemo se samo na kratko na osnovne mehanizme kriptografije koji se koriste u blokčejn sistemima.

Prvo uvodimo pojam jednosmerne (*hash*) funkcije. Jednosmerna funkcija je funkcija koja za ulaz prima podatke proizvoljne dužine, a vraća podatak fiksne dužine. Pošto slika beskonačan skup ulaza na konačan skup izlaza, jednosmerna funkcija nije injektivna, što znači da nema inverznu, tj. da se od dobijene slike ne može rekreirati original. Jednosmerne funkcije imaju dodatne karakteristike:

- Determinističnost – isti ulaz funkcija uvek slika na isti izlaz
- Uniformnost – funkcija ravnomerno mapira ulaze na skup izlaza
- Osetljivost na početne uslove – za malu promenu ulaza funkcija daje veliku promenu izlaza

Jednosmerne funkcije u blokčejnu se koriste u digitalnim potpisima, kao i za ulančavanje blokova (više o lancu blokova biće reči u kasnijim poglavljima).

Sada uvodimo termine vezane za šifrovanje poruka. Postupak šifrovanja u literaturi se naziva *enkripcija*, i koristi tajni podatak – ključ. Šifrovana poruka se zatim šalje primaocu. Ako se poruka presretne pri slanju, presretnik ne može da pročita sadržinu poruke. Kada primalac dobije šifrovanu poruku, pre nego je pročita on je mora dešifrovati – *dekriptovati*. Kao i šifrovanje i dešifrovanje koristi ključ. U opštem slučaju, ova dva ključa mogu biti različiti. Algoritmi koji koriste iste ključeve pri enkripciji i dekripciji nazivaju se simetričnim, dok se algoritmi koji koriste razne ključeve nazivaju asimetričnim. Za potrebe blokčejna asimetrična kriptografija ima mnogo značajniju ulogu te ćemo njoj posvetiti dodatno objašnjenje.

Asimetrični algoritmi kriptografije, kao što je već navedeno, koriste različite ključeve za proces enkripcije i dekripcije. Kako bi se izbegla potreba da svaki par učesnika deli poseban tajni ključ, razvijen je sistem javnih i tajnih ključeva. Sistem funkcioniše tako što svaki učesnik u komunikaciji

ima par ključeva – jedan za enkripciju i drugi za dekripciju. Ključ za enkripciju je javni i svi mogu da ga vide. Ključ za dekripciju, pak, ostaje tajan, i njega zna samo vlasnik. Ova pravila omogućuju svim zainteresovanim stranama da enkriptuju i pošalju poruku vlasniku, ali samo vlasniku da je dekriptuje.

Asimetrična kriptografija se takođe koristi u svrhu digitalnih potpisa. Kada se koristi u ovu svrhu, ključevi menjaju ulogu, ključ za enkripciju postaje tajan dok ključ za dekripciju postaje javan. Kada se neki dokument digitalno potpisuje prvo se određenom jednosmernom funkcijom ceo dokument hešira. Zatim se tako dobijeni heš enkriptuje (sad privatnim ključem). Ovako dobijeni potpis šalje se zajedno sa dokumentom. Prilikom provere potpisa, strana koja ga proverava opet hešira ceo dokument, potom javnim ključem vlasnika dekriptuje potpis, i upoređuje te dve heš vrednosti. Ako su vrednosti jedanke (iste) strana koja proverava može biti sigurna da dokumentom nije manipulirano. Ovaj postupak obezbeđuje autentifikaciju dokumenata pod sledećim uslovima: strani koja proverava je poznata korišćena heš funkcija (po konvenciji) i javni ključ vlasnika, a tajni ključ je poznat samo vlasniku.

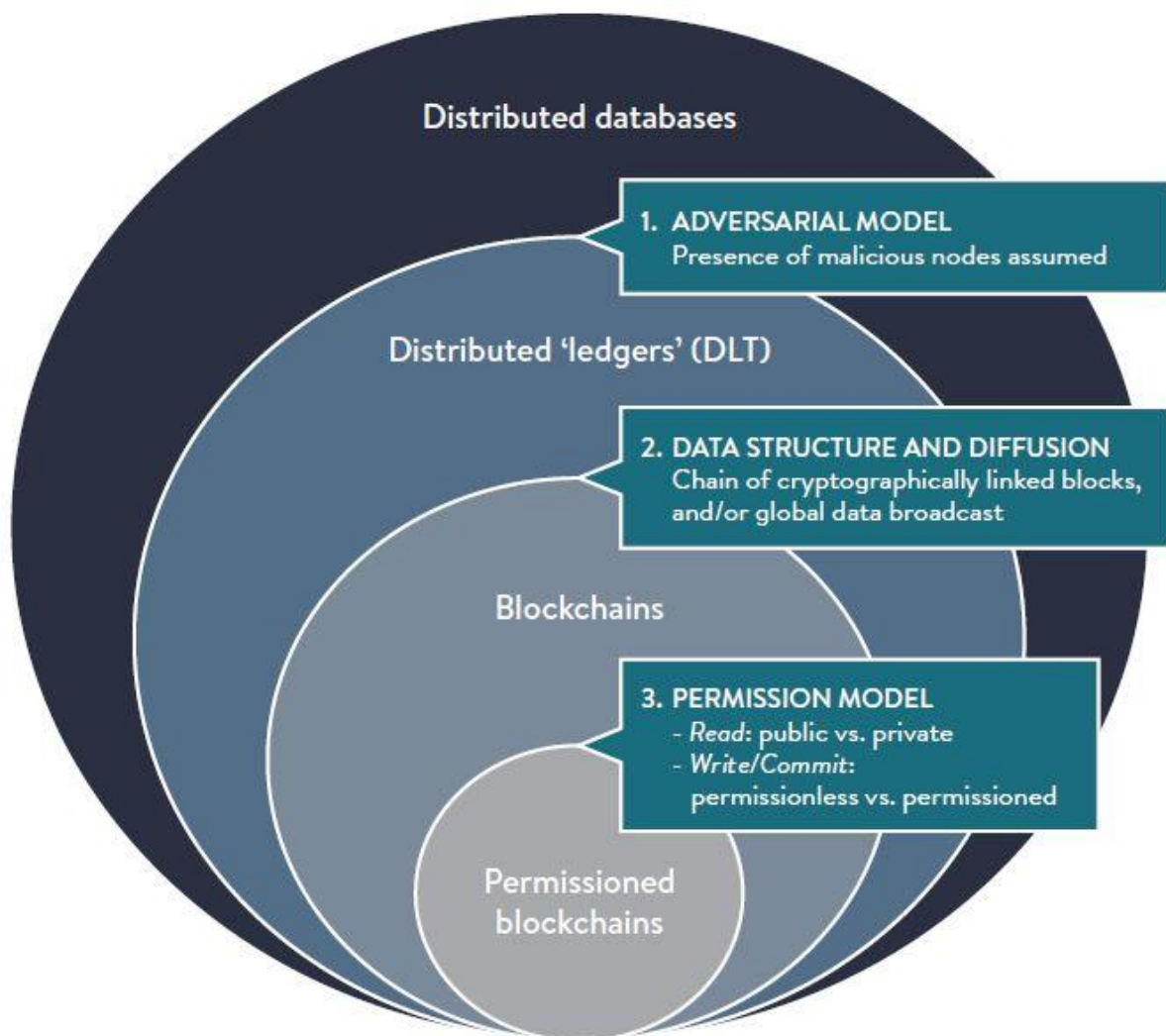
Digitalni potpisi se koriste prilikom slanja transakcija u blokčejnu, kako bi se osigurali integritet i neporecivost transakcija.

2.3 Blokčejn

Blokčejn kao tehnologija začeta je 2008. godine u radu anonimnog autora (ili grupe) pod pseudonimom Satoshi Nakamoto. Bila je prvenstveno namnjena, kao što je i naslov rada kaže², za elektronski sistem plaćanja. Od tada do danas, ova tehnologija doživela je veliki uspon i njena namena nije više isključivo, mada i dalje jeste primarno, u sistemima elektronskog plaćanja. Ovaj sistem se ne oslanja na centralizovani autoritet za validiranje transakcija, već je za validiranje koristi distribuiran algoritam konsenzusa. Blokčejn je podvrsta *tehnologija distribuirane glavne knjige* (*Distributed Ledger Tehnology* - DLT), koja je sama podvrsta distribuiranih baza podataka, kao što možete videti na slici 2.2.

² S. Nakamoto Bitcoin - Bitcoin: A Peer-to-Peer Electronic Cash System

Blockchains and distributed ledgers are types of distributed databases



Source: Global Blockchain Benchmarking Study, by Dr Garrick Hileman & Michel Rauchs, 2017 - University of Cambridge
via @neigrando

Slika 2.2 Veza između distribuiranih baza podataka, DTL-ova i blokčejna
(peruzeto iz [22])

U ovom radu će se pojam blokčejn upotrebljavati kao krovni termin za celokupni sistem i njegove mehanizme, dok će se sama struktura podataka nazivati lanac blokova. Dublji upliv u strukturu lanca blokova dat je u poglavljima koja slede.

Treba napomenuti da će opis funkcionisanja sistema biti u najvećoj meri analogan Bitcoin-u, iz prostog razloga što on čini prvu generaciju blokčejna, tj. najprostiji je. Specifičnosti druge i treće generacije blokčejna objašnjene su u podglavljima na kraju.

2.3.1 Problem

Ideja o decentralizovanom sistemu elektronskog plaćanja mnogo je starija od Bitcoin-a. Međutim, postojao je problem koji do 2008. niko nije znao kako da reši. Svi prethodni pokušaji

pravljenja distribuiranog bankovnog sistema nisu uspjeli da onemoguće zlonamernim korisnicima da svoj novac višestruko troše. Da bismo razumeli ovaj problem moramo dati definiciju elektronskog (digitalnog) novčića.

Elektronski novčić (*coin*) je definisan kao niz digitalnih potpisa [9]. Svaki korisnik u sistemu digitalnog bankarstva ima svoj javni i privatni ključ, koji se koriste za potpisivanje transakcija. Kada platilac želi da obavi plaćanje digitalnim novčićem on javni ključ primaoca, kao i vreme uplate enkriptuje (potpisuje) svojim privatnim ključem. Na ovaj način, primalac može da se uveri, dekriptovanjem (korišćenjem javnog ključa platioca) potpisa, da je novčić zaista „promenio“ vlasnika. Ovaj način prenosa vlasništva nad novčićem, pak, dozvoljava zlonamernom korisniku da isti novčić upotrebi i za drugo plaćanje, tj. potpiše javni ključ drugog primaoca i drugi datum uplate na isti niz prethodnih potpisa. Efektivno govoreći, tada je zlonamerni korisnik potrošio svoj novac dva puta, stoga i naziv problema dvostruka potrošnja (*double spend*).

Preteče Bitcoin-a nisu uspjeli da pronađu rešenje ovog problema pomoću distribuiranog algoritma pa su pribegavali korišćenju centralnog autoriteta koji je vodio evidenciju o validnosti transakcija. Naime, svi novčići bi se slali centralnom autoritetu koji bi potom skovao novi novčić i poslao ga primaocu. Samo novčići direktno dobijeni od centralnog autoriteta smatrali bi se validnim. Ovakva realizacija je u velikoj meri analogna bankama u fizičkom svetu, i ne zaobilazi problem poverenja centralnog autoriteta. Za rešenje problema potreban je distribuiran algoritam koji obezbeđuje da se novčići ne potroše više puta [9].

2.3.2 DLT

Sad kad razumemo uslove pod kojima je nastao i problem koji pokušava da reši, možemo preći na opis funkcionisanja blokčejna. Rešenje problema dvostruke potrošnje jeste vođenje evidencije, ali na distribuiran način, tako da se svi čvorovi (učesnici sistema) međusobno dogovaraju koje transakcije jesu, a koje nisu validne i time određuju novo stanje sistema (stanje sveta), to i jeste definicija DLT-a. Kao što je već rečeno, blokčejn je podtip DLT-a. To znači da će svaki (puni) čvor voditi zasebno evidenciju svih sredstava i dogovarati se oko novog stanja sistema algoritmom konsenzusa. Ono što blokčejn sisteme čini posebnim u okviru DLT-ova jeste da svaki (puni) čvor vodi i evidenciju promena kroz strukturu podataka koja predstavlja lanac blokova, s toga i naziv blokčejn (*chain of blocks - blockchain*). Da bismo razumeli kako funkcioniše struktura lanca blokova moramo odrediti kako izgleda, tj. od kojih se podataka sastoji, jedan konkretan blok.

2.3.3 Struktura bloka

Svaki blok unutar lanca sastoji se od svog tela i zaglavlja. U telu bloka nalaze se transakcije koje se u datom bloku validiraju, dok se u zaglavlju bloka nalaze različiti metapodaci.

2.3.3.1 Postupak validiranja transakcija

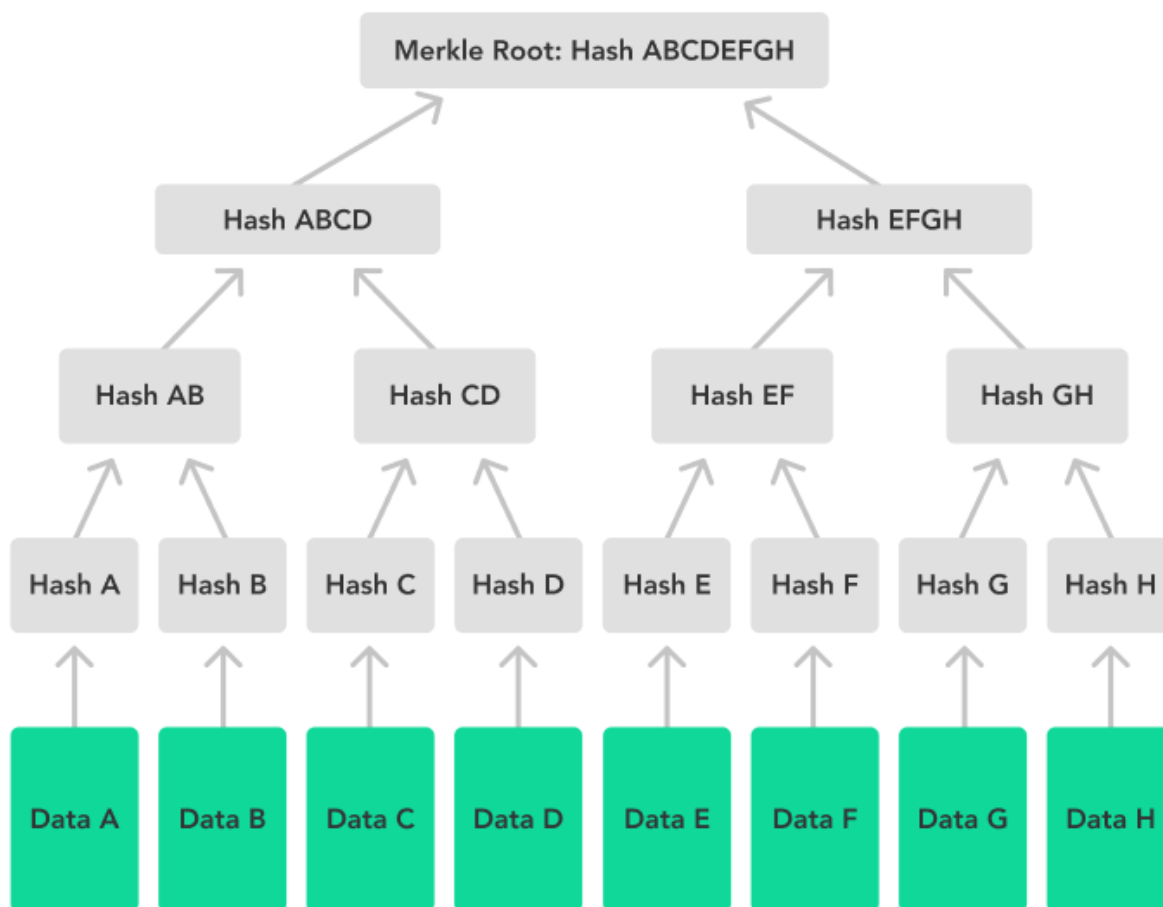
Da bi se način funkcionisanja sistema bolje opisao, u ovom delu je opisan put transakcije od korisnika sistema do pravljenja bloka. Napominjemo da će ovaj postupak sadržati samo prvi deo pravljenja bloka, što uključuje njegovu validaciju, popunjavanje tela bloka i većine zaglavlja, ali ne i objavljivanje bloka ostalim čvorovima, te algoritam konsenzusa. Ti delovi postupka pravljenja bloka detaljno su opisani u podpoglavljima koja slede.

Transakciju potpisuje platilac koristeći svoj privatni ključ, te je šalje sistemu na verifikaciju. Platilac ne šalje transakciju svim punim čvorovima, već samo jednom, najbližem, a on će dalje proslediti datu transakciju ostalima. U ovom trenutku pretpostavićemo da je sistem poravnat u svakom trenutku (da svi čvorovi vide sve transakcije). Videćemo kasnije da ova pretpostavka nije tehnički ispravna (i ne može biti, jer zbog veličine sistema mora da se primeni konačan model konzistentnosi), ali je za sada korisna. Dakle kad transakcija dođe do čvora ona biva smeštena u bazen transakcija sa ostalim transakcijama koje čekaju validaciju. Iz razloga koji će postati očigledni kasnije, čvorovi ne validiraju transakcije hronološkim redosledom. Kada čvor odabere transakcije koje će validirati u okviru sledećeg bloka (obično je broj transakcija eksponent dvojke), on kreće redom i validira pojedinačne transakcije. Kako svaki čvor ima evidenciju stanja sistema, ovaj postupak je trivijalan. Kada završi sa validacijom transakcija, čvor počinje da pravi strukturu podataka koja se zove *Merkleovo stablo*. Merkleovo stablo će detaljnije biti objašnjeno u nastavku izlaganja. Koren Merkleovog stabla se pohranjuje u zaglavlje bloka. Pored korena Merkleovog stabla, u zaglavlje se upisuju i vreme kreiranja bloka kao i heš vrednost prethodnog bloka. Ova logička veza čini datu strukturu lancem. Treba obratiti pažnju da heš vrednost prethodnog čini sastavni deo poslednjeg bloka, te utiče na njegov heš. Na ovaj način se prethodni blok, kao i svi blokovi pre njega, dodatno konsoliduju unutar lanca.

2.3.3.2 Merkleovo stablo

Merkleovo stablo čini posebnu vrstu punog binarnog stabla. Stablo ima broj listova jednak broju transakcija koje se validiraju. U listove stabla se pohranjuju heš vrednosti pojedinačnih transakcija. Svi dalji nivoi, počevši od prethodnog se nakon toga popunjavaju heš vrednošću kombinacije svojih dvaju potomaka. Ovaj postupak se ponavlja dok se ne stigne do korena stabla. Ilustracija Merkleovog stabla data je na slici ispod.

Merkle Tree With Eight Leaves



Slika 2.3 Merkleovo stablo

(preuzeto sa <https://www.ccn.com/education/crypto/bitcoin-merkle-trees-ensure-transaction-integrity/>)

Ovako izračunat koren se zapisuje u zaglavlje bloka. Razlog pravljenja ovakve strukture jeste verifikacija transakcija korisnika sistema. Kada korisnik želi, po pravljenju celog bloka, da se uveri da je njegova transakcija bila deo datog bloka, on, znajući heš vrednost transakcije koju proverava, od sistema dobija heš vrednost njemu susedne transakcije. Na ta taj način, korisnik može da rekreira roditeljski čvor datog lista. Osim susdenog lista svoje transakcije, korisnik dobija i heš vrednosti svih susednih čvorova svih svojih predaka. Na analogan način korisnik dakle može da rekreira koren Merkleovog stabla i da ga uporedi sa onim zapisanim u zaglavlju, uveravajući se da je njegova transakcija zaista deo datog bloka. Ovakav način verifikacije je izuzetno efikasan, pošto koristi samo $\log_2(n)$ proveru, gde je n broj transakcija u datom bloku.

2.3.4 Algoritmi konsenzusa

Pošto smo objasnili postupak pravljenja tela i dela zaglavlja bloka, u ovom delu ćemo uvesti problem konsenzusa i algoritme koje ga rešavaju. Takođe ćemo videti kako određeni algoritmi

konsenzusa zahtevaju dodatna polja u zaglavlju bloka. Na kraju je dat opis tri algoritma konsenzusa.

2.3.4.1 Otkazi i konsenzus

Distribuirani sistemi su inherentno podložni otkazima. Uprkos otkazima, distribuirani sistem treba da se automatski oporavi i nastavi sa radom. Za takve sisteme kažemo da su otporni na otkaze. Lamport je [3] opisao vrstu otkaza koja je naknadno, po datom problemu, dobila naziv vizantijski otkaz. Naime, vizantijski otkaz je onaj kod kog komponenta sistema različitim posmatračima pruža različite simptome otkaza. Svi distribuirani sistemi su podložni vizantijskom otkazu, ali se njegov značaj razlikuje u zavisnosti od tipa systemske arhitekture.

U centralizovanom sistemu otkaz centralnog autoriteta (servera) je svakako najnepovoljniji. Iako se dešava da centralni autoritet otkáže, njemu se, *a priori*, veruje, te se njegove odluke i direktive ne proveravaju. Takođe, centralnom autoritetu se veruje da nije maliciozan te se vizantijski otkazi smatraju posledicama grešaka u sistemu. Iz navedenih razloga problem vizantijskih generala je manje značajan u centralizovanim sistemima.

U decentralizovanom (P2P) sistemu, zbog nepostojanja centralnog autoriteta učesnici sistema moraju da međusobnim dogovorom dođu do odluke po raznim pitanjima. Učesnici P2P sistema zbog prirode sistema imaju ograničeno poverenje jedni u druge. Tako se u ovakvim sistemima teško da razlučiti između slučajnog otkaza i zlonamernog ponašanja, što problem dodatno ističe.

2.3.4.2 Problem vizantijskih generala

Problem vizantijskih generala tema je mnogobrojnih naučnih radova, tako da će u ovom podpoglavlju biti objašnjen samo ukratko. Za detaljniji opis svakako se preporučuje [3].

Problem je postavljen apstraktno. Vizantijska vojska opseda određeni grad. Vojska je podeljena u n delova, svaki od kojih je pod komandom određenog generala. Jedan od generala je glavni - komandant, dok su mu ostali podređeni – poručnici. Oni međusobno treba da se dogovore da li će napasti grad ili se povući. Začkoljica je u tome što među generalima može biti i izdajnika koji žele da sabotiraju opsadu. Izdajnici sabotiraju opsadu tako što različitim generalima šalju različite odluke. Cilj je naći način (algoritam) da se generali dogovore oko plana, tako da svi iskreni generali postupe isto. U radu su razmotrene dve situacije: sa usmenim i sa potpisanim porukama.

U verziji problema sa usmenim porukama ne postoji način da se sazna ko je poslao koju poruku, tako izdajica može da tvdi da mu je komandant poslao poruku različitu od zapravo poslate. Ovo u velikoj meri otežava problem, jer se ne može lako utvrditi ko je izdajica. Tako u ovoj varijanti

problema, da bi iskreni generali mogli da dođu do odgovora treba da čine više od dvotrećinske većine ($n = 3m + 1$, m – broj izdajnika).

U drugoj verziji, poruke se šalju potpisane, sa pretpostavkom da potpis ne može da se kopira. Dodatno, generali imaju podrazumevano ponašanje kome će pribeći ako shvate da ima izdajica. U ovakvoj situaciji, dokazano je da bilo koji broj iskrenih generala može da dođe do dogovora nezavisno od broja izdajica. Ova karakteristika direktna je posledica potpisivanja poruka, tj. nemogućnosti izdajica da neometano promene tuđe poruke.

Iz ovog problema nastao je veliki broj algoritama, najznačajiniji od njih je praktična vizantijska tolerancija otkaza (*Practical Byzantine Fault Tolerance* - PBFT). Za više informacija čitalac se upućuje na [5].

2.3.4.3 Konsenzus u DLT-u

Kao što je već navedeno DLT je vrsta distribuirane baze podataka, P2P mreža repilika koje se dogovaraju oko promene stanja sistema. Najvažnije odluke oko koje moraju da se slože jesu validnost i redosled obrade transakcija. Zato je u DLT-ovima biranje adekvatnog algoritma konsenzusa ključno.

Blokčejn je, kao što je takođe navedeno, podvrsta DLT-a, što znači da je pitanje konsenzusa ključno i za njegovo funkcionisanje. Neki algoritmi zbog svojih karakteristika, nisu pogodni za implementaciju u blokčejnu. U ovom trenutku treba se setiti pretpostavke koju smo uveli u podpoglavlju 2.3.3.1. da su svi čvorovi u mreži poravnati u svakom trenutku. Ova pretpostavka je bila korisno pojednostavljeno u datom trenutku, ali je sada ukidamo. Zbog veličine sistema, komunikacija između čvorova u mreži ima nezanemarljivo kašnjenje. Takođe, zbog velikog, nepoznatog i promenljivog broja članova mreže, čvorovi ne mogu komunicirati direktno svaki sa svakim, inače bi sistem trošio više vremena u komunikaciji nego u korisnom radu. To znači da centralizovani algoritmi, kao što je PBFT nisu adekvatni za ovu primenu.

2.3.4.4 Konsenzus u blokčejnu

Nemajući poverenja u čvorove sistema koji vrše validaciju, algoritam konsenzusa u blokčejnu služi kako bi se (ostali) čvorovi sistema dogovorili da li je blok koji je predložen validan ili nije. Svaki algoritam ima svoje implementacione detalje koji su specifični za dati algoritam, međutim svi imaju određene zajedničke osobine. Algoritam konsenzusa treba da zahteva od čvorova koji validiraju određeni ulog (bio on u digitalnoj valuti ili u fizičkom resursu). Ako se dati blok usvoji, čvor koji ga je predložio treba da dobije određenu nagradu, tj. ako se odbije treba da snosi određenu kaznu (gubitak uložених sredstava). Nagrada za kreiranje validnog bloka, u gotovo svim algoritmima, podrazumeva kovanje novog novčića i/ili uzminje provizije od validiranih

transakcija (ideja Bitcoin-a je da vremenom novi novčići prestanu da se kuju, a da se validatori nagrađuju ekskluzivno od provizije, što bi značilo uspostavljanje sistema bez dalje inflacije). U ovom trenutku postaje očigledno zašto validatori ne potvrđuju transakcije hronološkim redosledom, već u zavisnosti od toga koja transakcija će validatoru doneti najveću zaradu.

U nastavku izlaganja data su objašnjenja tri algoritma konsenzusa koji se koriste u blokčejnu. Prvi je najznačajniji, i najšire rasprostranjen, ali ima velike ekološke mane. Drugi je, po mnogima, zamena za prvi. Treći algoritam je mnogo manje poznat, ali je u ovom delu dat, više radi raznovrsnosti i zanimljive ideje, nego radi njegovog značaja u praksi.

2.3.4.5 Dokaz posla

Dokaz posla (Proof of Work - PoW) je algoritam konsenzusa koji je predložen (i usvojen) u Nakatomovom radu iz 2008. godine (iako je ideja proistekla ranije, videti [7]). Još uvek predstavlja najznačajniji algoritam konsenzusa. Dokaz posla se bazira na zakonu verovatnoće. Svaki blok u zaglavlju, pored prethodno opisanih polja, sadrži i polje koje se naziva *nonce* (engl. number used once - *nonce*). *Nonce* se generiše kao slučajan broj, potom se ceo blok (duplo) hešira i vrednost tako dobijenog heša treba da zadovolji određeni kriterijum. Kriterijum koji se koristi na Bitcoin mreži jeste da prvih n bita binarne reprezentacije datog heša treba da budu nule. Broj zahtevanih počenih nula bita se čuva u zaglavlju bloka, on se ponderiše kako bi se uspostavila projektovana dinamika validiranja blokova (10 min/blok). Povećavanjem n , eksponencijalno se smanjuje opseg odgovarajućih heševa. Karakteristike heš funkcija garantuju da se iz odgovarajućeg heša ne može inverznim postupkom dobiti traženi *nonce*, što validate primorava da u petlji pokušavaju sa mnogo različitih vrednosti, ne bi li slučajno našli odgovarajuću vrednost. Ove mnogobrojne operacije heširanja sa velikim brojem različitih vrednosti *nonce*-a, troše mnogo električne energije, što čini trošak validatorima. Kada bi zlonamerni validator predložio blok koji nema zadovoljavajući heš bio bi odmah odbijen, a provera zahteva samo jednu operaciju heširanja. Ako validator predloži blok koji sadrži nevalidnu transakciju, a koji ima zadovoljavajuću heš vrednost, ostali validatori bi, po proveru transakcija unutar bloka, odbili dati blok. Zlonamerni validator u ovom slučaju ipak snosi trošak cene električne energije utrošene zarad računanja heš vrednosti.

Ako validator predloži korektan blok, i ostatak mreže ga usvoji, on dobija nagradu u vidu provizije od validiranih transakcija i pravo pravljenja novog novčića.

Distribuirani sistemi sa velikom količinom čvorova, podataka i korisnika ne mogu biti konzistentni u svakom trenutku bez velikih posledica na performanse sistema. Tako, u stvarnosti, različiti čvorovi sistema mogu u isto vreme da imaju različite podatke. To znači da je moguća situacija da više validatora u približno isto vreme predloži različite blokove. U ovom slučaju, čvorovi imaju konvenciju da prihvataju najduži lanac blokova, dok ostale odbacuju. Kako po

predlaganju blokova, svi konkurentni lanci imaju istu dužinu $m+1$ (ako je m bila dužina lanca bez novog bloka), različiti delovi mreže će raditi sa različitim blokovima. Koji od ovih lanaca će biti izabran za glavnog, zavisi od toga kom lancu će pripasti sledeći validiran blok. U trenutku validacije novog bloka, jedan od datih lanaca ima dužinu $m+2$, dok ostali imaju dužinu $m+1$, te će se ostali lanci odbaciti. U praksi, zbog velike distribuiranosti mreže i kašnjenja u komunikaciji, dešava se da glavni lanac mora da ima više od dva nova bloka kako bi bio sigurno izabran za glavnog (neka nezvanična konvencija je da lanac treba da ima bar šest blokova kako bi bilo sigurno da će biti izabran za glavnog).

Ovaj algoritam je osetljiv na takozvane 51% napade. Ako jedan zlonameran validator, ili grupa validatora, kontroliše 51% računarske moći ona može, uz dovoljno vremena, izgenerisati najduži lanac i time efektivno promeniti stanje sistema. Napadač ne može prisvojiti tuđa sredstva, pošto to ostali validatori ne bi prihvatili, ali može da povрати svoja (vraćamo se na problem dvostruke potrošnje). Ova strategija napada ima svoje nedostatke. Prvo, sa 51% računarske moći, napadač bi, i uz iskreno ponašanje, zarađivao, što predstavlja podsticaj validatorima da se ponašaju iskreno. Drugo, zlonamerni validator svojim napadima narušava integritet mreže, u koju ima uložena novčana sredstva, što neminovno smanjuje vrednost njegovih sredstava.

Još jedna mana ovog algoritma je što veliki broj validatora vrši ogroman broj računarskih operacija koji ne doprinose direktno sistemu, što troši veliku količinu električne energije. Neka procena iz 2018. godine je da se zbog „rudarenja“ Bitcoin-a godišnje potroši 71.12 TWh električne energije, slično godišnjoj potrošnji Čilea [6]. Zbog ovog negativnog uticaja na okolinu, počela je da se traži zamena za PoW algoritam. 2022. godine Ethereum mreža je počela da koristi PoS algoritam.

2.3.4.6 Dokaz uloga

Dokaz uloga (*Proof of Stake* - PoS) je algoritam koji je zamenio PoW na Ethereum mreži. PoS ne koristi *nonce* polje i ne računa heš bloka u potrazi za odgovarajućom vrednosti. Umesto toga, validatori direktno ulažu svoja novčana sredstva, u formi kriptovalute. Nakon faze ulaganja sledi izbor validatora koji će predložiti novi blok. Validator se bira slučajno, a verovatnoća da će biti izabran je proporcionalna količini uložених sredstava (generisanje slučajnih brojeva u blokčejnu nije trivijalna operacija, i predstavlja poseban problem koji prevazilazi okvire ovog rada). Slučajnost pri izboru je važna karakteristika algoritma jer ona osigurava da najbogatiji učesnik mreže ne bude uvek izabran da kreira novi blok. Jednom izabran, validator predlaže blok koji ostali validatori pregledaju. Ako se slože da ga prihvate, nagradu za kreiranje bloka dele svi validatori, dok onaj validator koji ga je predložio dobija najveći deo. Ako u toku glasanja validatori pokažu maliciozno ponašanje, koje predstavlja glasanje suprotno većini, tada im se oduzima ulog

i oni gube pravo da vrše validaciju u budućnosti (*slashing*). Takođe, ako izabrani validator predloži nekorektan blok, njemu će se oduzeti ulog. Najveća prednost algoritma dokaza uloga nad dokazom posla je što ne troši ni približno mnogo električne energije, te je mnogo ekološki prihvatljiviji. Pored toga, ovaj algoritam nije osetljiv na 51% napade.

2.3.4.7 Dokaz proteklog vremena

Dokaz proteklog vremena (*Proof of Elapsed Time* - PoET) je algoritam konsenzusa koji je proedložio Intel u svom Hyperledger Sawtooth-u. Za razliku od prethodnih algoritama, koji se izvršavaju na javnim blokčejn mrežama, PoET se izvršava na privatnim blokčejn mrežama (detaljniji opis privatnih blokčejn mreža dat je u poglavljima koja slede). Zbog ove razlike PoET ne definiše direktno kaznu za predlaganje nevalidnih blokova, već samo način raspoređivanja prava na pravljenje novog bloka između validatora sistema. Oslanja se na posebne hardverske komponente, tzv. pouzdana okruženja za izvršavanje (*Trusted Execution Environment* - TEE). Uloga TEE-a je da generiše slučajan broj koji predstavlja vreme spavanja. Nakon što je proteklo dobijeno vreme čvor se budi i, ako nijedan čvor nije probuđen pre njega, kreira novi blok. Na ovaj način čvorovi simuliraju proces rudarenja spavanjem, bez utroška električne energije. Ovaj algoritam sličan je algoritmu dokaza sreće (*Proof of Luck*)[16].

2.3.5 Generacije blokčejna

Prva generacija blokčejna začeta je 2008. godine već pomenutim radom Nakamoto-a. Kako je većina prethodnog teorijskog opisa bila zasnovana na njemu, on u nastavku nije dalje razmotren. Za ostale specifičnosti Bitcoina čitalac se upućuje na [9].

2.3.5.1 Blokčejn II generacije

Kako je Bitcoin postajao popularniji, tako su ljudi počeli da ga koriste kao prekrivajuću mrežu za druge namene (pr. *colorcoin*). Videvši ovaj trend, Vitalik Buterin dobio je ideju pravljenja blokčejn mreže koja bi bila pogodnija kao prekrivajuća mreža i za koju bi bilo lakše pisati aplikacije. Iako je i Bitcoin mreža imala programski jezik koji je koristila – Script (koji u ranijem izlaganju nije pomenut zbog svog relativno malog značaja), on je bio domenski specifičan. Bio je osmišljen za finansijske transakcije sa ciljem da bude izuzetno bezbedan. Zato Script nije Turing-kompletna (ne sadrži petlje). Novi blokčejn je trebalo da sadrži virtualnu mašinu i Turing-kompletna jezik koji bi se na njoj izvršavao. Ova mreža nazvana je *Eterium* (*Etherium*), jezik je nazvan *ethereum bytecode*, virtualna mašina *EVM* (*Etherium Virtual Machine*), a valuta na njoj *etar* (*ether*). Kako je *ethereum bytecode* jezik niskog nivoa apstrakcije (asemblerki jezik zasnovan na steku, sličan Java bytecode-u), uz njega osmišljena su i četiri jezika visokog nivoa apstrakcije koja se prevode na njega, od kojih je najznačajniji *Solidity* (detaljniji opis *Solidity*-a dat je u

nastavku rada). Glavna prednost Eteriuma je što je bio pogodan za pisanje *pametnih ugovora* – aplikacija koje se automatski izvršavaju na blokčejnu. Ovi ugovori mogu služiti kao digitalni novčići, kao distribuirane organizacije (*Decentralized Autonomous Organizations* - DAOs) ili u druge svrhe.

Iako se nećemo mnogo udubiti u organizaciju same mreže, za suštinu ovog rada važno je napomenuti nekoliko detalja. Da bi se izbegle slučajne ili (zlo)namerne eksploatacije problema koji Turing-kompletni jezik sa sobom donosi – beskonačne petlje, beskonačne rekurzije, uveden je sistem gasa. Svaka instrukcija u *bytecode*-u ima određenu cenu izraženu u gasu (kaže se „troši x gasa“), i pozivalac funkcije pametnog ugovora pri samom pozivu određuje maksimalnu količinu gasa koju je spreman da plati za izvršavanje date funkcije. Ako navedena maksimalna količina gasa zadovoljava potrebe funkcije, funkcija će se izvršiti nesmetano, a višak gasa biće vraćen pozivaocu. Ako, pak, pozivalac ne nameni dovoljnu količinu gasa, po isteku sveg gasa, bilo kakve eventualne promene stanja sistema napravljene u toku rada funkcije biće vraćene, a gas se ne refundira. Takođe gas nema unapred definisanu cenu, već cenu određuje pozivalac funkcije pri samom pozivu. Više cene gasa podstiču validatore da uključe datu transakciju (poziv funkcije je i dalje transakcija) u novi blok (daju joj prioritet). Ovaj mehanizam ipak utiče na skalabilnost mreže. Sa povećanjem broja korisnika povećava se i broj transakcija koje oni žele da izvrše. Sa druge strane, brzina validiranja blokova se nepovećava. Ove dve činjenice *per se* nisu sporne, ali kad su prisutne zajedno nastaje problem: veći broj transakcija treba da se validira u približno jednakom broju blokova, što podstiče korisnike da povećaju cenu gasa, ne bi li im se transakcija brže validirala, iliti drugim rečima, podstiče inflaciju. Da bi se zaobišli problemi vezani za ovako dobijenu inflaciju, Eterium je počeo da realizuje mreže drugog sloja (*Layer 2 Network* – L2). Ove mreže funkcionišu analogno glavnoj, koriste istu virtualnu mašinu i koriste etar kao kriptovalutu. Ideja jeste da se svakodnevni promet odvija na L2 mrežama, a da se sredstva povremeno vrata na glavnu mrežu. Kako je L2 mreža ipak autonomna od glavne, prenos sredstava sa jedne na drugu mora se odvijati preko posrednika. Program koji posreduje i omogućuje automatski prenos sredstava sa glavne na L2 mrežu i obrnuto naziva se *most (bridge)*. Prototip mosta, njegova softverska arhitektura i realizacija dati su u četvrtom poglavlju.

2.3.5.2 Blokčejn III generacije

Dosad razmatrani Bitcoin i Eterium bili su javne mreže, što znači da bilo koji računar može da se uključi u mrežu i da učestvuje u njoj. Međutim, za blokčejn tehnologiju (i DLT generalno) su se vremenom nalazilale i druge primene. Različite organizacije (korporacije) uvidele su da međusobnu saradnju, uz delimično ali ne potpuno poverenje jedna u drugu, mogu da ostvare koristeći blokčejn tehnologiju. Tako je nastala ideja privatne blokčejn mreže, koja se zasniva na

ograničavanju učešća samo za autentifikovane, autorizovane članove. Ovakve mreže zbog delimičnog poverenja između svojih članova, ne zahtevaju tako rigidan algoritam konsenzusa poput PoW ili PoS. U ovim sistemima primenu su našli algoritmi poput spomenutih PoET ili PoL. Sama organizacija (sistemska arhitektura) ovakve mreža zbog svoje namene mnogo je varijabilnija nego Bitcoin ili Eterium. Kako bi se privatne blokčejn mreže lakše razvijale Linux fondacija je pokrenula krovni projekat Hyperledger, koji se sastojao od više manjih projekata. Najznačajniji među njima su Hyperledger Fabric i spomenuti Hyperledger Sawtooth. Oni predstavljaju radne okvire (*framework*) – softverske platforme koje omogućuju i ubrzavaju razvoj privatnih blokčejn mreža. Najčešće koriste programski jezik Go(lang).

3 Opis korišćenih tehnologija

Za implementaciju ovog projekta korišćene su sledeće tehnologije: programski jezik Solidity (za pisanje pametnih ugovora), Ganache (za simuliranje blokčejna), programske biblioteke web3.py i web3.js (za interakciju sa blokčejnom), python biblioteka flask (za implementaciju back-end dela web aplikacije) i javascript biblioteka react (za implementaciju front-end dela veb aplikacije).

3.1 Programski jezik Solidity

Programski jezik Solidity je nastao 2014. godine. Njegov tvorac je Gavin Vud, koosnivač Ethereum-a. Jezik je osmišljen za pisanje pametnih ugovora na Ethereum platformi, prevodi se u *bytecode* zarad izvršavanja na EVM-u ili kompatibilnim virtualnim mašinama. Na njegov razvoj uticaj su imali programski jezici C++, python i javascript. Jezik je statički tipiziran, objekto-orijetnisan jezik. Pametni ugovori su analogni klasama (u nastavku će se pojmovi klase i ugovora koristiti kao sinonimi), podržavaju nasleđivanje, imaju konstruktor, polja i metode (funkcije).

Vrste memorije u programskom jeziku Solidity su: *memory*, *storage* i *calldata*. *Storage* je trajna memorija i njoj pripadaju sva polja klase. *Memory* je memorija koja traje za vreme poziva određene funkcije. *Calldata* služi za obeležavanje memorije zauzete za argumente poziva funkcija.

Ugovor se, prilikom isporuke, inicijalizuje konstruktorom. Sva dalja interakcija sa ugovorom odvija se kroz mehanizme poziva funkcija i događaja. Događaj karakterišu naziv, vreme okidanja i argumenti okidanja, i oni su mehanizam kojim pametni ugovori daju povratne informacije spoljašnosti sistema.

Srž pametnih ugovora su funkcije koje mogu biti pozvane u cilju opsluživanja. Funkcije imaju modifikatore pristupa. Funkcije koje mogu biti pozvane izvan ili unutar ugovora obeležavaju se ključnom rečju *public*. Funkcije koje su dostupne kao podrutine drugih funkcija obeležavaju se ključnom rečju *private*. Funkcije dostupne samo spoljašnosti sistema označene su kao *external*. Funkcija može takođe biti obeležena ključnim rečima *view* ili *pure*. *View* označava funkciju koja ne menja stanje blokčejna, zato je njen poziv uvek jeftiniji od funkcije koja to radi. *Pure* funkcije su podvrsta *view* funkcija – osim što ne menjaju stanje blokčejna, one ni ne intereaguju sa njim, zbog toga su imenovane kao čiste.

Polja klase mogu biti označena kao *public*, međutim ovakva deklaracija će samo izgenerisati privatno polje i funkciju označenu kao *external view*, kojom se to polje može očitati, ali ne i promeniti.

Funkcije mogu biti označene kao *payable*. To su funkcije prilikom čijih poziva pozivalac, osim troškova izvršenja funkcije, može preneti određena sredstva na račun (adresu) pozivanog ugovora.

Funkcije mogu imati određena ograničenja koja se odnose na pristup funkcijama. Na primer, neretka je situacija u kojoj želimo da ograničimo pristup funkciji samo na vlasnika ugovora. Tada bi se informacija o vlasniku sačuvala u okviru konstruktora i proveravala se pre izvršenja funkcije. Za tu proveru u Solidity-ju se koriste *require* iskazi. Kako nekad logika provere može biti kompleksna (zahtevati više *require* iskaza) i može se ponavljati, Solidity podržava mogućnost grupisanja provera u dekoratorske funkcije - modifikatore.

Pozivanje funkcija i reagovanje na događaje su detaljnije pojašnjeni u delu izlaganja o softverskom rešenju.

3.2 Ganache

Ganache je lokalni privatni blokčejn namenjen razvoju i testiranju aplikacija na Ethereum platformi. Omogućava programerima da brzo pokreću blokčejn okruženje na svom računaru, bez potrebe za povezivanjem sa javnim mrežama, čime se ubrzava razvoj i testiranje pametnih ugovora.

Ganache nudi rad i preko komandne linije i putem grafičkog interfejsa (GUI), što omogućava jednostavno upravljanje blokovima, transakcijama i nalogima. Podržava kreiranje i simultano pokretanje više blokčejn instanci, čime se olakšava testiranje aplikacija u različitim scenarijima.

Posebno je koristan jer emulira stvarne Ethereum funkcionalnosti, uključujući transakcije, rudarstvo blokova i događaje pametnih ugovora, ali u kontrolisanom i predvidivom okruženju. Pruža potpun uvid u stanje mreže, balanse naloga i tok transakcija, što znatno olakšava razvoj, debugovanje i validaciju aplikacija pre njihove implementacije na javni Ethereum.

3.3 Programske biblioteke web3.py i web3.js

Programske biblioteke web3.py i web3.js su biblioteke za rad sa blokčejnom u jezicima python i javascript, respektivno. Kako je jedina razlika između korišćena dve biblioteke konvencija nazivanja funkcija (web3.py koristi podrazumevanu konvenciju za python – zmijsku notaciju, dok web3.js koristi podrazumevanu konvenciju za javascript – kamilju notaciju), razmatraćemo samo verziju biblioteke za python programski jezik.

Biblioteka web3.py omogućava dvosmernu komunikaciju sa blokčejn mrežom kroz standardizovan interfejs. Omogućava upravljanje Ethereum čvorovima, rad sa nalogima i novčanicima, slanje i potpisivanje transakcija, kao i interakciju sa pametnim ugovorima putem

njihovih ABI (*Application Binary Interface*) definicija. Takođe pruža mehanizme za praćenje događaja i promene stanja na blokčejnu u realnom vremenu.

Ova biblioteka je u projektu korišćena za pisanje nove biblioteke, koja pojednostavljuje rad sa blokčejnom uvodeći viši nivo apstrakcije.

3.4 Flask

Flask je mikro veb okvir (*framework*) za programski jezik python, namenjen razvoju serverskih (*back-end*) aplikacija i REST API servisa.

Razvijen je u zajednici otvorenog koda, a odlikuje se jednostavnošću, fleksibilnošću i proširivošću. Zahvaljujući modularnoj arhitekturi, omogućava programerima da po potrebi dodaju komponente za rad sa bazama podataka, autentifikaciju, šablonima i drugim funkcionalnostima. Flask se često koristi u projektima gde su potrebni stabilnost, mala složenost i brza implementacija serverske logike.

3.5 React

React je javascript biblioteka za razvoj *front-end* aplikacija zasnovanih na komponentama. Osmišljen je u Facebook-u, 2010. godine, da bi kasnije postao dostupan kao biblioteka otvorenog koda. Oslanja se na komponente koje imaju visok stepen ponovne iskoristivosti. Obrazuje virtuelno DOM stablo i njime manipuliše, te ažurira DOM (*Document Object Model*) stablo u pretraživaču po potrebi. Ova tehnika eliminiše nepotrebna osvežavanja i time poboljšava performanse aplikacije. Danas je *de facto* standard u industriji.

4 Opis softverskog rešenja

Opis rešenja je podeljen na nekoliko celina: izgled korisničkog interfejsa, softverska arhitektura rešenja, pametan ugovor i najvažnije funkcionalnosti *back-end* aplikacije. *Front-end* aplikacija zbog svoje jednostavnosti i sličnosti između biblioteka za rad sa blokčejnom ne predstavlja okosnicu projekta, te je u ovom delu njen opis izostavljen kao trivijalan.

4.1 Arhitektura sistema

Arhitektura ovog sistema je hibridna. Jedan deo se nalazi na blokčejnu (pametni ugovori), a drugi predstavlja ustavljenu troslojnu veb aplikaciju. Sloj prezentacije podataka je realizovan kao *front-end* aplikacija koristeći React biblioteku. Ona komunicira sa *back-end* aplikacijom (sloj obrade) koja je realizovana pomoću Flask radnog okvira, koja čita podatke iz baze podataka (sloj perzistencije). Manipulacija korisničkim nalogima i provizijama realizovana je u sloju obrade, ali ovaj sloj sme samo da čita transakcije iz baze. Zasebni procesi služe za osluškivanje blokčejnova i rukovanje uplatama, te rukovanje njihovim zaključivanjem.

4.2 Pametan ugovor

Most u ovom projektu je dizajniran da spaja, kao što je ranije navedeno, glavnu mrežu sa mrežom na drugom nivou (*Ethereum mainnet* i L2 mrežu). Kako je cena etra na L2 mreži uslovljena njegovom cenom na glavnoj mreži, to se projektant odlučio da sistem funkcioniše bez kovanja tokena. U ovom trenutku je važno napomenuti da ovo pojednostavljenje ne važi u opštem slučaju, tj. između bilo koje dve mreže. Ugovor je simetričan, u smislu da se isti ugovor izvršava na obe mreže.

Zbog značaja ugovora u nastavku je dat u celosti, ali je zbog veličine datoteke, razlomljen na nekoliko celina.

```
//SPDX-License-Identifier: MIT

pragma solidity ^0.8.0;

struct PendingTransaction {
    uint256 id;
    address sender;
    address recipient;
    uint256 ammount;
    uint32 client_id;
}

struct PaymentInfo {
    uint256 id;
    address recipient;
    uint256 ammount;
}
```

Listing 4.1 Pomoćne strukture

Na početku datoteke definisane su dve strukture: *PendingTransaction* i *PaymentInfo*. Prva sadrži polja za identifikator, adresu pošiljaoca (na trenutnoj mreži), adresu primaoca (na drugoj mreži), uplaćeni iznos i identifikator klijenta. Ova struktura služi kako bi se prenele vitalne informacije o uplati sa blokčejna na *back-end*.

Druga sadrži polja samo za identifikator, adresu primaoca i iznos za uplatu primaocu. Ova struktura služi kako bi se skladištili nalozi za isplatu koji stižu sa *back-end*-a. Voditi računa da se polja iznosa u dve strukture razlikuju za odbijenu proviziju.

```
struct QueueTransaction {
    mapping( uint => PendingTransaction ) transactions;
    uint128 first;
    uint128 last;
}

function qt_enqueue(QueueTransaction storage queue, PendingTransaction memory
new_transaction) {
    require(queue.last + 1 > queue.last, "Queue full!");
    queue.transactions[++queue.last] = new_transaction;
    if (queue.first == 0){
        queue.first = queue.last;
    }
}
```

Listing 4.2 Red nad strukturom *QueueTransaction* i funkcija dodavanja

```

function qt_dequeue(QueueTransaction storage queue)
returns(PendingTransaction memory){
    PendingTransaction memory transaction = queue.transactions[queue.first];
    delete queue.transactions[queue.first++];
    if (queue.first > queue.last){
        queue.first = queue.last = 0;
    }
    return transaction;
}

function qt_get_count(QueueTransaction storage queue) view returns(uint){
    if (queue.first == 0){
        return 0;
    }
    return queue.last - queue.first + 1;
}

```

Listing 4.3 Funkcije za skidanje sa reda i vraćanje broja elemenata u redu

Kako u programskom jeziku *Solidity* ne postoji ugrađena podrška za strukturu reda, osnovne funkcionalnosti neophodne za njenu implementaciju date su u listinzima 4.2 i 4.3. U njima je red implementiran preko rečnika (mapiranja), kao kružni bafer, vodeći računa o prvom i poslednjem elementu. Analogna struktura i funkcije postoje i nad strukturom *PaymentInfo* ali nisu prikazane.

```

contract Bridge {
    address                private _owner;
    uint256                private _id;
    uint16                private _step;
    uint256                private _min_value;
    QueueTransaction       private _pending;
    QueuePayment           private _wired;

    event new_deposit(uint256 latest_id);
    event retrieved(uint256 id, address sender , address recipient,
        uint256 ammount, uint32 client_id);
    event wired(uint256 id, address recipient, uint256 ammount);
    event payout(uint256 id, uint256 ammount);

    constructor (uint256 start_id, uint16 step, uint256 min_value) {
        _owner      = msg.sender;
        _id         = start_id;
        _step       = step;
        _min_value  = min_value;
    }
}

```

Listing 4.4 Polja pametnog ugovora, događaji i konstruktor

Na listingu 4.4 vide se sva polja ugovora, događaji i konstruktor. Polja označavaju vlasnika, identifikator prethodne uplate koju je ugovor primio, korak povećanja, minimalna vrednost uplate

i redove uplata i isplata. Događaji u sistemu su sledeći: nova uplata, preuzeta uplata, transakcija je poslata na isplatu i isplata izvršena.

```

modifier only_owner {
    require(msg.sender == _owner, "[FAIL] Authorization failure. Only owner
permitted.");
    _;
}

function see_min_value() external view returns(uint256){
    return _min_value;
}

function get_id() private returns(uint256) {
    _id += _step;
    return _id;
}

function check_wired(uint256 id) external view returns(int) {
    return qp_check(_wired, id);
}

```

Listing 4.5 Modifikator za pristup vlasnika, pomoćne funkcije

Na Listingu 4.5 prikazan je modifikator koji pristup funkciji dozvoljava samo vlasniku ugovora, pomoćne funkcije za dobavljanje identifikatora, minimalne uplate i provere da li je transakcija sa traženim identifikatorom stigla u red za isplatu.

```

function settle_transaction() private returns(bool) {
    if (qp_get_count(_wired) == 0)
        return false;
    PaymentInfo memory latest_transaction = qp_dequeue(_wired);

    if (latest_transaction.ammount > address(this).balance)
        return false;
    payable(latest_transaction.recipient).transfer(
latest_transaction.ammount);

    emit payout(latest_transaction.id, latest_transaction.ammount);
    return (qp_get_count(_wired) > 0);
}

function settle_all_posible() private {
    while (settle_transaction()) {}
}

```

Listing 4.6 Funkcije za isplatu

Na listingu 4.6 prikazane su funkcije za isplatu. Funkcija *settle_all_possible* služi da isplati sve transakcije redom, staje kad nema više isplata u redu ili se potrošio sav etar sa računa ugovora.

```

function inquiry() only_owner external view returns(uint256) {
    return address(this).balance;
}

function owner_withdrawal(uint256 ammount) only_owner external payable {
    require(address(this).balance > ammount, "[FAIL] Ammount exceeds the
balance.");
    payable(_owner).transfer(ammount);
}

function owner_deposit() only_owner external payable {
    settle_all_posible();
}

```

Listing 4.7 Pomoćne vlasničke funkcije

Listing 4.7 prikazuje pomoćne funkcije za proveru sredstava na računu ugovora, dopunu i eventualno isplatu sredstava sa računa ugovora, kako bi se izbegao problem nestanka sredstava sa računa ugovora.

```

function deposit(address recipient, uint32 client_id) external payable{
    require(msg.value > _min_value, "[FAIL] Deposit has to be greater. To see
minimum access function see_min_value");
    uint256 id = get_id();
    PendingTransaction memory new_transaction = PendingTransaction(id,
msg.sender, recipient, msg.value, client_id);
    qt_enqueue(_pending, new_transaction);
    emit new_deposit(id);
}

function retrieve_transaction() only_owner external {
    require(qt_get_count(_pending) > 0, "[FAIL] No transactions pending");
    PendingTransaction memory last_transaction = qt_dequeue(_pending);
    emit retrieved(last_transaction.id, last_transaction.sender,
last_transaction.recipient, last_transaction.ammount,
last_transaction.client_id);
    settle_all_posible();
}

```

Listing 4.8 Glavne funkcije uplate

Listing 4.8 prikazuje glavne funkcije uplate. Prva je funkcija koju poziva klijent kako bi pokrenuo prenos sredstava. Uplata dobija identifikator i stavlja se u red uplata, zatim se emituje događaj uplate.

Funkcija za dobavljanje uplata poziva *back-end* kao reakciju na okinut događaj uplate, služi da se dobave informacije o uplati. One se emituju kroz događaj, kako bi *back-end* aplikacija mogla da ih pročita. Budući da je legla uplata na račun ugovora, isplate koje su eventualno bile blokirane

nedostatkom sredstava sada možda mogu biti sprovedene. Zato se na kraju funkcije poziva rutina za isplatu.

Takođe, treba uzeti u razmatranje da je sva ova logika mogla da se implementira unutar prethodne funkcije. Izdvajanje funkcije za dobavljanje uplate od funkcije za uplatu služi dve svrhe. Prvo, mogućnost rada ugovora i kad *back-end* eventualno otkáže, pa će transakcije biti prenete kad se server ponovo pokrene. Drugo, red za isplatu može biti proizvoljno dugačak, i nije uputno da pozivalac funkcije uplate (klijent) snosi trošak cene gasa za tuđu isplatu.

```
function wire(uint256 id, address recipient, uint256 ammount) only_owner
external{
    PaymentInfo memory new_transaction = PaymentInfo(id, recipient, ammount);
    qp_enqueue(_wired, new_transaction);
    emit wired(new_transaction.id, new_transaction.recipient,
new_transaction.ammount);
    settle_all_posible();
}
```

Listing 4.9 Funkcija za slanje transakcije na isplatu

U listingu 4.9 vidimo funkciju koju *back-end* poziva kako bi preneo informacije o isplati na drugu mrežu. Događaj prenosa se emituje kako bi ostao trag o prebacivanju naloga, zatim se poziva rutina za isplatu. Treba obratiti pažnju da iz prethodnih listinga proizilazi da se isplate obrađuju hronološki, što znači da manje isplate mogu biti blokirane većim u slučaju da na računu ugovora ima dovoljno sredstava za isplatu manjeg, ali ne i većeg naloga, koji mu prethodi.

4.3 *Back-end* aplikacija

Back-end aplikacija se satoji iz dva dela: procesi za rukovanje pametnim ugovorom i *REST API* server.

4.3.1 Rukovanje pametnim ugovorom

Za potrebe olakšanja interakcije sa pametnim ugovorom razvijene su male, pomoćne funkcije, koje obrazuju mini probramsku biblioteku.

```
class Account:
    address      :str
    private_key  :str
    def __init__(self, address:str, private_key:str):
        self.address      = address
        self.private_key  = private_key
```

Listing 4.10 Struktura naloga


```

class BlockchainInfo:
    web_address :str
    port        :int
    chain_id    :int
    def __init__(self, web_address:str, port:int, chain_id:int):
        self.web_address = web_address
        self.port        = port
        self.chain_id    = chain_id

class ContractBlueprint:
    abi        : list
    bytecode   : dict
    def __init__(self, abi:list, bytecode:dict):
        self.abi        = abi
        self.bytecode   = bytecode

```

Listing 4.11 Strukture blokčejna i šeme ugovora

```

def execute_transaction(blockchain: BlockchainInfo, account: Account,
    target_function, value: int = 0) -> dict:
    try:
        proxy            = get_blockchain_connection(blockchain)
        nonce            = get_nonce(proxy, account)
        transaction      = build_transaction(blockchain, account, nonce,
target_function, value)
        signed_trasaction = proxy.eth.account.sign_transaction(transaction,
private_key=account.private_key)
        tx_hash          =
proxy.eth.send_raw_transaction(signed_trasaction.raw_transaction)
        tx_receipt       = proxy.eth.wait_for_transaction_receipt(tx_hash)
        return tx_receipt
    except ContractLogicError as e:
        print(colored(f'[EXCEPTION] {e.args[1]['reason']}', 'red'))
        return None
    except Exception as e:
        print(colored(f'[EXCEPTION] {e}', 'red'))
        return None

```

Listing 4.12 Funkcija za izvršavanje proizvoljne transakcije

Na listingu 4.12 prikazana je funkcija za izvršavanje proizvoljne transakcije (poziva proizvoljne funkcije). Prvo se uspostavi veza sa mrežom, potom se dobavi *nonce*, transakcija se izgradi, potom se potpiše, pošalje na izvršenje i sačeka se račun. *Nonce* polje je različito za svaku transakciju, kako maliciozan učesnik ne bi mogao da ponovi prethodnu transakciju.

```

def get_blockchain_connection(blockchain: BlockchainInfo):
    return
Web3(Web3.HTTPProvider(f'http://{blockchain.web_address}:{blockchain.port}'))

```

Listing 4.13 Pomoćna funkcija za uspostavljanje veze ka mreži

```
def get_nonce(proxy: Web3, account: Account) -> int:
    return proxy.eth.get_transaction_count(account.address)
```

Listing 4.14 Pomoćna funkcija za dobavljanje nonce-a

```
def build_transaction(blockchain: BlockchainInfo, account: Account, nonce: int,
    unbuilt_transaction, value:int = 0):
    transaction = unbuilt_transaction.build_transaction({
        'chainId' : blockchain.chain_id,
        'from' : account.address,
        'nonce' : nonce,
        'value' : value
    })
    return transaction
```

Listing 4.15 Pomoćna funkcija za izgradnju transakcije

```
def listen_for_deposit(network_1: Network, network_2: Network,
    connection_string_builder: IConnectionStringBuilder, mail_sender: MailSender,
    poll_interval:int = 3) -> None:
    global shutdown_event
    network_1.init_contract()
    network_2.init_contract()
    event_filter_n1 =
network_1._contract.events.new_deposit.create_filter(from_block='latest')
    event_filter_n2 =
network_2._contract.events.new_deposit.create_filter(from_block='latest')
    repository = get_repository_base(connection_string_builder)
    transaction_service = TransactionService(repository)
    commission_service = CommissionService(repository)
    user_service = UserService(repository)
    while not shutdown_event.is_set():
        try:
            handle_deposits(network_1, network_2, transaction_service,
                commission_service, user_service, mail_sender, event_filter_n1)
            handle_deposits(network_2, network_1, transaction_service,
                commission_service, user_service, mail_sender, event_filter_n2)
            time.sleep(poll_interval)
        except KeyboardInterrupt as e:
            shutdown_background('SIGNINT')
            continue
        except Exception as e:
            print(colored(f'[EXCEPTION] Background process recieved an unexpected
exception. Exiting... {str(e)}', 'red'))
            break
    close_session(repository)
    print(colored(f'[INFO] Process {current_process().pid} exiting...', 'blue'))
```

Listing 4.16 Funkcija koja osluškuje za nove uplate

Na listinzima 4.16, 4.17 i 4.18 prikazane su glavne funkcije za prebacivanje sredstava sa jedne mreže na drugu. Glavna funkcija se periodično okine i prozove mreže ne bi li dobila informacije o novim događajima. Ukoliko se događaj desio, uplate se obrađuju dok se identifikator

update ne izjednači sa brojem poslednje uplate (dobijenim iz događaja). Obrada se vrši u funkciji prikazanoj na listgu 4.18. Uplate se čitaju iz bloka, čuvaju se u bazi podataka, odbija im se provizija, a zatim se šalju na izvršenje. Na kraju se šalje e-mail sa potvrdom pošiljaocu.

```
def handle_deposits(network_origin: Network, network_destination: Network,
transaction_service: TransactionService, commission_service: CommissionService,
user_service: UserService, mail_sender: MailSender, event_filter) -> None:
    proxy = network_origin.get_blockchain_connection()

    for event in event_filter.get_new_entries():
        print(colored(f'[INFO] Deposit detected: {network_origin._name} ====>
{network_destination._name}', 'blue'))
        try:
            tx_hash = event['transactionHash']
            tx_receipt = proxy.eth.get_transaction_receipt(tx_hash)
            recent_id = find_most_recent_id(network_origin, tx_receipt)
            while True:
                max_id = retrieve_and_wire(network_origin, network_destination,
transaction_service, commission_service, user_service,
mail_sender)
                if max_id == recent_id:
                    break
        except Exception as e:
            print(colored(f'[EXCEPTION] {e}', 'red'))
            continue
```

Listing 4.17 Funkcija za rukovanje uplatama

```
def retrieve_and_wire(network_origin : Network, network_destination: Network,
transaction_service: TransactionService, commission_service: CommissionService,
user_service: UserService, mail_sender: MailSender) -> int:
    transactions = network_origin.retrieve_transactions()
    save_transactions(transactions, transaction_service, commission_service,
user_service)
    transactions = deduct_commission(transactions, transaction_service,
commission_service)
    max_id = max([x._id for x in transactions])
    for transaction in transactions:
        tx_receipt = network_destination.wire(transaction)
        wired_data = network_destination.get_event_data(tx_receipt, 'wired')
        change_state_wire(wired_data, transaction_service)
        send_mail_wired(mail_sender, user_service, transactions)
    return max_id
```

Listing 4.18 Funkcija za prenos sredstava na drugu mrežu

```

def listen_for_settlement(network_1: Network, network_2: Network,
connection_string_builder: IConnectionStringBuilder, mail_sender: MailSender,
poll_interval:int = 3) -> None:
    global shutdown_event
    network_1.init_contract()
    network_2.init_contract()
    event_filter_n1 =
network_1._contract.events.payout.create_filter(from_block='latest')
    event_filter_n2 =
network_2._contract.events.payout.create_filter(from_block='latest')
    repository = get_repository_base(connection_string_builder)
    transaction_service = TransactionService(repository)
    user_service = UserService(repository)

    while not shutdown_event.is_set():
        try:
            handle_settlements(network_1, transaction_service, user_service,
connection_string_builder, mail_sender, event_filter_n1)
            handle_settlements(network_2, transaction_service, user_service,
connection_string_builder, mail_sender, event_filter_n2)
            time.sleep(poll_interval)
        except KeyboardInterrupt as e:
            shutdown_background('SIGNINT')
            continue
        except Exception as e:
            print(type(e))
            print(colored(f'[EXCEPTION] Background process recieved an unexpected
exception. Exiting... {str(e)}', 'red'))
            break
    close_session(repository)
    print(colored(f'[INFO] Process {current_process().pid} exiting...', 'blue'))

```

Listing 4.19 Funkcija koja osluškuje za isplate

Na listinzima 4.19 i 4.20 prikazane su funkcije koje osluškuju mreže za izvršenje isplata. Kad detektuje isplatu, funkcija *handle_sattlements* pozove rutinu *sattle* koja promeni stanje transakcije na završena i pošalje mejl sa tom informacijom pošiljaocu.

```

def handle_settlements(network: Network, transaction_service: TransactionService,
user_service: UserService, connection_string_builder: IConnectionStringBuilder,
mail_sender: MailSender, event_filter) -> None:
    proxy = network.get_blockchain_connection()

    for event in event_filter.get_new_entries():
        print(colored(f'[INFO] Deposit settlement detected on network
{network._name}', 'blue'))
        try:
            tx_hash      = event['transactionHash']
            tx_receipt   = proxy.eth.get_transaction_receipt(tx_hash)
            transaction_service._repository =
replace_repository(transaction_service._repository, connection_string_builder)
            settled = settle(network, tx_receipt, transaction_service)
            send_mails_settled(mail_sender, user_service, settled)
        except Exception as e:
            print(colored(f'[EXCEPTION] {e}', 'red'))
            continue

```

Listing 4.20 Funkcija koja rukuje isplatama

4.3.2 Web server

```

def create_app(connection_string_builder: IConnectionStringBuilder,
session_configurator: SessionConfigurator, cors_configurator: CorsConfigurator,
admin_info:UserInfo) -> Flask:
    app = Flask(__name__)

    configure_session(app, session_configurator)
    configure_database_connection(app, connection_string_builder)
    configure_cors(app, cors_configurator)

    database.init_app(app)
    add_blueprints(app, database)
    Session(app)

    with app.app_context():
        database.create_all()
        check_and_create_admin(admin_info)

    return app

```

Listing 4.21 Funkcija koja pokreće web server

Na listingu 4.21 prikazana je funkcija koja pravi veb aplikaciju. Prvo se napravi Flask objekat, zatim se konfigurišu sesija, baza podataka i CORS politika (*Cross Origin Resource Sharing*), inicijalizuje se baza podataka, funkcije *add_blueprints* (videti listing 4.22) i *create_blueprints* (listing 4.23) prave sve neophodne pomoćne objekte, servise i kontrolere i dodaju ih u serverski objekat.

```
def add_blueprints(app: Flask, database: SQLAlchemy) -> None:
    repository = RepositoryBase(database.session)
    converter = JSONConverter()
    app.config['REPOSITORY'] = repository

    blueprints = create_blueprints(repository, converter)
    register_blueprints(app, blueprints)

    return
```

Listing 4.22 Funkcija za učitavanje konfiguracije

```
def create_blueprints(repository: IRepositoryBase, converter: IJSONConverter) ->
List[Blueprint]:

    commission_service = CommissionService(repository)
    transaction_service = TransactionService(repository)
    user_service = UserService(repository)

    commission_controller = create_commission_controller(commission_service,
converter)
    transaction_controller = create_transaction_controller(transaction_service,
converter)
    user_controller = create_user_controller(user_service, converter)

    blueprints:List[Blueprint] = [commission_controller, transaction_controller,
user_controller]
    return blueprints
```

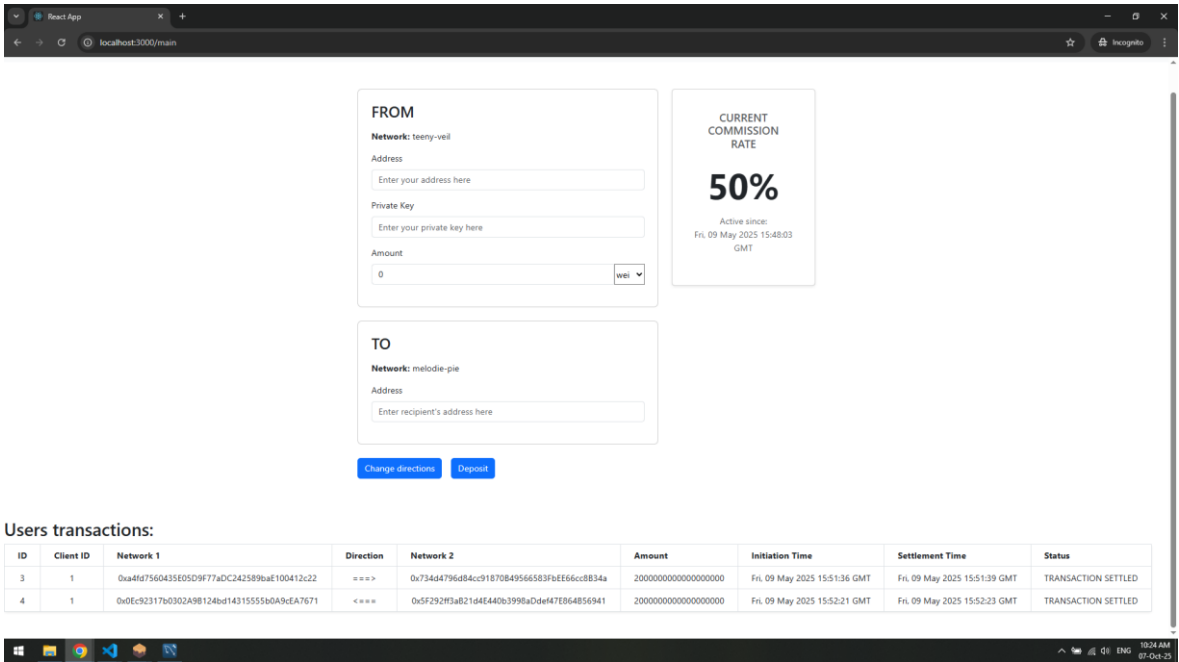
Listing 4.23 Funkcija koja inicijalizuje kontrolere

4.4 Izgled korisničkog interfejsa

Nakon uspešnog prijavljivanja na sistem, u zavisnosti od uloge, korisnika dočekuje jedna od dve stranice: administratorska ili korisnička.

Korisnička stranica je jednostavna (videti sliku 4.1): na njoj se nalazi centralna forma na prenos sredstava sa jednog blokčejna na drugi i tabela sa prethodnim transakcijama.

Forma se sastoji od polja za unos adrese računa pošiljaoca, njegovog privatnog ključa i iznosa za prenos. Ispod se unosi adresa računa primaoca (na drugom blokčejnu). Pored forme se nalazi kartica sa procentom provizije koju sistem naplaćuje, kao i informacija o periodu aktivnosti date provizije. Ispod forme se nalazi tabela sa transakcijama prijavljenog korisnika. Svaka transakcija sadrži identifikator, identifikator klijenta, adrese računa na dva blokčejna, smer prenosa, iznos koji se preneo, datum i vreme slanja i obrade i status transakcije.



Slika 4.1 Izgled korisničke stranice

Commissions

ID	Commence Time	Expiry Time	Percentage
1	Tue, 01 Apr 2025 18:35:14 GMT	Tue, 01 Apr 2025 18:36:38 GMT	2.7 %
2	Tue, 01 Apr 2025 18:36:38 GMT	Wed, 02 Apr 2025 15:58:24 GMT	0.5 %
3	Wed, 02 Apr 2025 15:58:24 GMT	Wed, 02 Apr 2025 16:36:19 GMT	0.4 %
4	Wed, 02 Apr 2025 16:36:19 GMT	Tue, 08 Apr 2025 16:07:26 GMT	0.3 %
5	Tue, 08 Apr 2025 16:07:26 GMT	Wed, 16 Apr 2025 16:46:28 GMT	50 %
6	Wed, 16 Apr 2025 16:46:28 GMT	Wed, 16 Apr 2025 16:46:52 GMT	2.4 %
7	Wed, 16 Apr 2025 16:46:52 GMT	Wed, 23 Apr 2025 20:48:09 GMT	3 %
8	Wed, 23 Apr 2025 20:48:09 GMT	Wed, 30 Apr 2025 16:12:16 GMT	3.5 %
9	Wed, 30 Apr 2025 16:12:16 GMT	Fri, 09 May 2025 15:48:03 GMT	4 %
10	Fri, 09 May 2025 15:48:03 GMT	ACTIVE	50 %

Set commission: [Change Commission](#)

Transactions

ID	Client ID	Network 1	Direction	Network 2	Amount	Initiation Time	Settlement Time	Status
3	1	0xa4fd7560435E05D9F77aDC242589baE100412c22	==>	0x734d4796d84cc91870B49566583fbEE66cc8B34a	20000000000000000000	Fri, 09 May 2025 15:51:36 GMT	Fri, 09 May 2025 15:51:39 GMT	TRANSACTION SETTLED
4	1	0x0Ec92317b0302A9B124bd14315555b0A9cEA7671	<==	0x5F292f3aB21d4E440b3998aDdef47E864B56941	20000000000000000000	Fri, 09 May 2025 15:52:21 GMT	Fri, 09 May 2025 15:52:23 GMT	TRANSACTION SETTLED

Slika 4.2 Izgled administratorske stranice

Deo administratorske stranice prikazan je na slici 4.2. Na njemu se vidi tabela provizija i tabela svih transakcija. Provizije su izlistane hronološki, svaka ima procenat provizije koji se naplaćuje, identifikator i datum i vreme početka i kraja aktivnosti. Ispod tabele se nalazi polje za promenu provizije.

Tabela transakcija ima ista polja kao klijentska, ali sadrži transakcije svih klijenata.

5 Zaključak

Most predstavljen u ovom radu je tek prototip, i ne bi mogao komercijalno da se upotrebi. Neka ograničenja bi trebalo da se prevaziđu pre nego što bi ovaj prototip mogao biti konkurentan komercijalnim sistemima.

Prvo, korisnički interfejs bi trebalo da se obogati, da se administratoru dodaju opcije za proveru, uplatu i isplatu sredstava sa računa na obe mreže. Uvažavajući privatnost korisnika, trebalo bi omogućiti korisnicima da anonimno koriste usluge aplikacije, tj. da se omogući prenošenje sredstava bez obaveznog prijavljivanja. Takođe bi bilo uputno odvojiti polja za sredstva koje pošiljalac želi da uplati i potrebnu sumu (sa dodatom provizijom).

Drugo, u svetu blokčejna, uopšteno, vlada nepoverenje u centralne autoritete i treća lica, zato bi bilo poželjno umesto zahtevanja privatnog ključa pošiljaoca uvesti podršku za kripto novčanike poput MetaMask-a.

Dodatno, pozadinski procesi koji rukuju prenošenjem sredstava bi trebalo da se optimizuju. Pošto je usko grlo sistema vreme između iskopavanja dva bloka, na šta ova aplikacija ne može da utiče, nije optimalno blokirati aplikaciju čekajući na izvršenje spoljnih događaja, već je predlog autora uvesti asinhronu obradu.

Implementacija sistema takođe ne prati u dovoljnoj meri obrasce prisutne u komercijalnim sistemima: potrebno je obogatiti logovanje, refaktorisati kod, automatizovati testove, obogatiti rukovanje greškama itd...

Poslednje, vreme prenosa sredstava je, čak i u najboljem slučaju, izuzetno dugo. Klijent mora da uplati novac na račun ugovora (1 blok), sistem mora da prepozna uplatu i da je očita (1 blok) i na kraju sredstva moraju biti prebačena na drugu mrežu (1 blok). Ovo znači da je za prebacivanje sredstava potrebno najmanje tri bloka, što uvodi relativno veliko kašnjenje. Prenos bi mogao da funkcioniše u dva bloka, što bi uštedelo dosta vremena.

Iako puno jednostavniji i manje opšt od komercijalnih sistema, autor smatra da je ovaj prototip koristan kao osnova za dalji razvoj i nadogradnju.

SKRAĆENICE

IBM	- <i>International Buiseness Machines</i>
CDN	- <i>Content Deliverz Network</i>
URL	- <i>Uniform Resource Locator</i>
DNS	- <i>Domain Name System</i>
IDL	- <i>Interface Definition Language</i>
LAN	- <i>Local Area Network</i>
GAN	- <i>Global Area Network</i>
SAN	- <i>Storage Area Network</i>
SETI	- <i>Search for ExtraTerrestrial Inteligance</i>
OMG	- <i>Object Management Group</i>
TCP	- <i>Transmission Control Protocol</i>
IP	- <i>Internet Protocol</i>
P2P	- <i>Peer 2 (to) Peer</i>
DLT	- <i>Distributed Ledger Technology</i>
PoW	- <i>Proof of Work</i>
PoS	- <i>Proof of Stake</i>
PoET	- <i>Proof of Ellapsed Time</i>
EVM	- <i>Etherium Virtual Machine</i>
DAO	- <i>Distributed Autonomous Organization</i>
ABI	- <i>Application Binary Interface</i>
DOM	- <i>Document Object Model</i>

6 Literatura

- [1] Andrew S. Tanenbaum, Maarten Van Steen: *Distributed Systems, 2nd Edition*, Prentice Hall, 2006
- [2] Leslie Lamport: *Time, Clocks, and the Ordering of Events in a Distributed System*, Communications of the ACM, 21(7):558–565, 1978
- [3] Leslie Lamport: *The Byzantine Generals Problem*, ACM Transactions on Programming Languages and Systems, 4(3):382–401, 1982
- [4] Leslie Lamport: *The Part-Time Parliament*, ACM Transactions on Computer Systems, 16(2):133–169, 1998
- [5] Miguel Castro, Barbara Liskov: *Practical Byzantine Fault Tolerance*, Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI), 173–186, 1999
- [6] José Luis Romero Ugarte: *Distributed Ledger Technology (DLT): Introduction*, Economic Bulletin, Banco de España, 2018
- [7] Cynthia Dwork, Moni Naor: *Pricing via Processing or Combatting Junk Mail*, Advances in Cryptology – CRYPTO ’92, Lecture Notes in Computer Science, vol. 740, Springer, 139–147, 1993
- [8] Ronald L. Rivest, Adi Shamir, Leonard Adleman: *A Method for Obtaining Digital Signatures and Public-Key Cryptosystems*, Communications of the ACM, 21(2):120–126, 1978
- [9] Satoshi Nakamoto: *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008, <https://bitcoin.org/bitcoin.pdf> [pristupljeno 08. oktobar 2025.]
- [10] Vitalik Buterin: *Ethereum White Paper: A Next Generation Smart Contract & Decentralized Application Platform*, 2013, https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf [pristupljeno 08. oktobar 2025.]
- [11] Mic Bowman, Debajyoti Das, Avradip Mandal, Hart Montgomery: *On Elapsed Time Consensus Protocols*, Advances in Cryptology – INDOCRYPT 2021, Lecture Notes in Computer Science, vol. 13143, Springer, 559–583, 2021
- [12] Miroslav Hajduković, Žarko Živanov: *Arhitektura računara (pregled principa i evolucije)*, Fakultet tehničkih nauka, Novi Sad, 2004. ISBN 978-86-80249-91-9
- [13] Wikipedia: *Beowulf cluster*, Wikipedia, https://en.wikipedia.org/wiki/Beowulf_cluster [pristupljeno 8. oktobar 2025.]

- [14] Dušan Gajić: *Materijali sa predmeta Paralelni i distribuirani algoritmi i strukture podataka*
- [15] Imre Lendak: *Materijali sa predmeta Osnove distribuiranog programiranja*
- [16] Mitar Milutinovic, Warren He, Howard Wu, Maxinder Kanwal: *Proof of Luck: An Efficient Blockchain Consensus Protocol*, IACR Cryptology ePrint Archive Report 2017/249, 2017
- [17] Solidity: *Solidity 0.8.30 documentation*, Solidity,
<https://docs.soliditylang.org/en/v0.8.30/> [pristupljeno 8. oktobar 2025.]
- [18] Truffle Suite: *Ganache Documentation*,
<https://trufflesuite.com/ganache/> [pristupljeno 8. oktobar 2025.]
- [19] Ethereum Foundation: *Web3.py Documentation*,
<https://web3py.readthedocs.io/en/stable/> [pristupljeno 8. oktobar 2025.]
- [20] Pallets Projects: *Flask Documentation*, *Flask*,
<https://flask.palletsprojects.com/en/latest/> [pristupljeno 8. oktobar 2025.]
- [21] Sebastijan Stoja: *Materijali sa predavanja Primena veb programiranja u infrastrukturnim sistemima*
- [22] Garrick Hileman, Michael Rauchs: *Global Blockchain: Benchmark Study*, Cambridge Centre for Alternative Finance, 2017

7 Biografija

Luka Đukić je rođen 2003. godine u Senti, gde je završio osnovnu školu *Stevan Sremac* i gimnaziju. Fakultet tehničkih nauka, studijski program primenjeno softversko inženjerstvo uspisuje 2021. i diplomira 2025. godine. Dalje akademsko i stručno usavršavanje planira u oblasti računarstva visokih performansi.